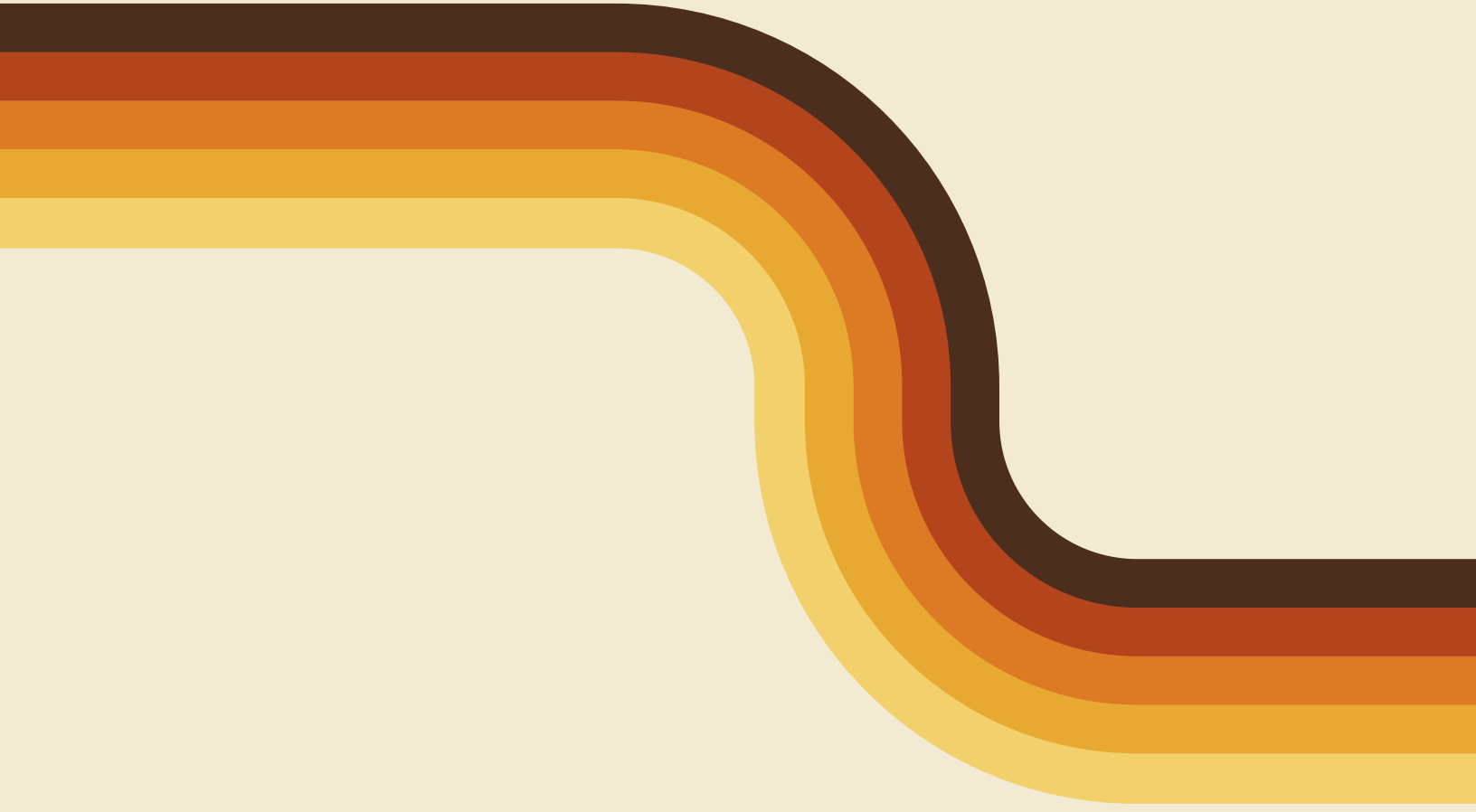
TFR

Manual and Reference

NitrOS-9 Level 2 on the TFR/911H



Order No. TFR-911H-MR
Version 1.4, October 2026

the tfr/9 project

TFR Manual and Reference

NitrOS-9 Level 2 on the TFR/911H: Commands, Special Files, Games, Networking, and Maintenance

This manual describes the commands, games, and tools added to NitrOS-9 Level 2 on the TFR/911H single board computer, the hardware the TFR/911H firmware presents to OS-9, networking with a FujiNet, and how to build and maintain the system. It begins with an introduction to OS-9 Level 1 and Level 2 as they run on the board.

Order Number:	TFR-911H-MR
Version:	1.4, 25 October 2026
Revision/Update Information:	Version 1.4: **tsh**, the shell with line editing.
Operating System and Version:	NitrOS-9/6809 Level 2 V3.3.0
Hardware and Firmware:	TFR/911H, firmware v4

TFR Manual and Reference, Version 1.4, 25 October 2026

Revision history

0.1	17 Oct 2026	First printing, as the *TFR/911H NitrOS-9 Reference Pages*.
0.2	17 Oct 2026	The FujiNet, and booting NitrOS-9 from it.
0.3	18 Oct 2026	Chapter 2, Networking; **fnnews** and **fnstocks**.
0.4	18 Oct 2026	fujinet-lib for NitrOS-9; colors.
1.0	18 Oct 2026	Named the *TFR Manual and Reference*, and versioned; the cover; **fnbounce**; UTF-8 in the tether.
1.1	18 Oct 2026	**fnweather** in C, with pictures, the week's forecast, and a cache; the first, as **fnwx**.
1.2	18 Oct 2026	**fnbattleship**: Fuji Battleship, in color and with sound.
1.3	20 Oct 2026	Release jamesBrown1, and the *TFR Installation Guide*; both of the FujiNet firmware's fixes are in fujinet-firmware itself.
1.4	25 Oct 2026	**tsh**, the shell with line editing, history, completion, and a search path; the firmware no longer loses a byte of a packet with 254 bytes that are not zero before a zero.

The information in this document is subject to change without notice and should not be construed as a commitment by anyone in particular. The software described in this document is furnished as source code, and may be used, copied, and changed under the terms of the licenses that come with it; the FujiNet programs, as the FujiNet project's are, under the GNU General Public License, version 3.

No responsibility is assumed for the use or reliability of software on equipment that was soldered together at a kitchen table.

© The TFR/9 Project contributors, 2026. All rights reserved.

Trademarks. OS-9 is a trademark of Microware Systems Corporation. NitrOS-9 is the work of the NitrOS-9 Project. Zork is a registered trademark of Infocom, Inc. Tetris is a trademark of The Tetris Company. Raspberry Pi is a trademark of Raspberry Pi Ltd. ESP32 is a trademark of Espressif Systems. Color Computer and CoCo are associated with Tandy Corporation. ULTRIX and DEC are trademarks of Digital Equipment Corporation.

About the design. This manual is typeset in the manner of Digital Equipment Corporation's ULTRIX reference pages of the early 1990s, as a tribute; its cover, in Futura, in the manner of a manual of the 1970s. It is not a Digital document, and it has nothing to do with Digital Equipment Corporation or its successors.

This document was prepared with LaTeX from `v4/nitros9-l2/doc/manual.tex`.

Contents

About This Manual iv

1 Introduction 1–1

1.1 The TFR/911H . . . 1–1
1.2 OS-9 Level 1 and Level 2 . . . 1–1
1.3 Level 1 on the TFR/911H: TurbOS . . . 1–2
1.4 Level 2 on the TFR/911H: NitrOS-9 V3.3.0 . . . 1–2
1.4.1 The machine NitrOS-9 sees . . . 1–2
1.4.2 The disk . . . 1–2
1.4.3 The modules . . . 1–3
1.5 What Was Added . . . 1–3
1.6 A First Session . . . 1–4

2 Networking 2–1

2.1 The FujiNet . . . 2–1
2.1.1 The hardware . . . 2–1
2.1.2 The firmware . . . 2–1
2.2 How the FujiNet Connects to the TFR/911H . . . 2–2
2.2.1 The wiring . . . 2–2
2.2.2 Inside the RP2350B . . . 2–2
2.2.3 Watching the link . . . 2–3
2.2.4 Booting from the FujiNet . . . 2–3
2.3 DriveWire . . . 2–3
2.4 The FujiNet in NitrOS-9 . . . 2–4
2.4.1 The modules . . . 2–4
2.4.2 Synchronous, with interrupts masked . . . 2–5
2.4.3 Level 1 and Level 2 . . . 2–5
2.5 Programming the FujiNet . . . 2–5
2.5.1 The SS.Fuji call . . . 2–6
2.5.2 Management commands . . . 2–6
2.5.3 The network device . . . 2–7
2.5.4 Building a program . . . 2–8
2.6 An Example: fnwx . . . 2–8
2.6.1 The plan . . . 2–9
2.6.2 Locating . . . 2–9
2.6.3 Fetching the weather . . . 2–9

2.6.4 Turning numbers into words . . . 2–10
2.6.5 The screen . . . 2–10
2.6.6 Writing your own . . . 2–10
2.7 Programs in C . . . 2–11
2.7.1 fujinet-lib for NitrOS-9 . . . 2–11
2.7.2 The terminal . . . 2–11
2.7.3 The five programs . . . 2–12
2.7.4 Building and testing . . . 2–13

3 Reference Pages 3–1

Section 1: Commands . . . 3–2
beep(1) *ring the bell* . . . 3–3
clear(1) *clear the screen* . . . 3–4
click(1) *turn keyclicks on or off* . . . 3–5
fnbattleship(1) *Fuji Battleship* . . . 3–6
fnbounce(1) *Bounce World* . . . 3–7
fnconfig(1) *FujiNet CONFIG* . . . 3–8
fnnews(1) *the news* . . . 3–9
fnstocks(1) *stock quotes* . . . 3–10
fnweather(1) *the weather where you are* . . . 3–11
fujinet(1) *commands for the FujiNet* . . . 3–12
grep(1) *print lines that contain a string* . . . 3–13
info(1) *show the system and the hardware* . . . 3–14
pwd(1) *print the data or execution directory* . . . 3–16
tsh(1) *the shell, with line editing* . . . 3–17
unix(1) *Unix-style names for OS-9 commands* . . . 3–20
vi(1) *a small vi* . . . 3–21
Section 4: Special Files . . . 3–23
fujinet(4) *the FujiNet* . . . 3–24
sn76489(4) *the sound chip* . . . 3–25
tfr911h(4) *the hardware the firmware emulates* . . . 3–26
Section 6: Games . . . 3–27
tetris(6) *Tetris, with music* . . . 3–28
zm(6) *a Z-machine, for Infocom's games* . . . 3–29
Section 8: Maintenance . . . 3–30
nitros9-l2(8) *build the system and keep a disk up to date* . . . 3–31
tcl(8) *the firmware's console* . . . 3–32
tether(8) *the board's terminal, disk, and clock* . . . 3–33

A Connecting a Speaker A–1

A.1 Wiring . . . A–1
A.2 Trying It . . . A–1
A.3 How It Works . . . A–1

B How the Software Was Tested **B–1**

B.1 zm B–1
B.2 vi B–1
B.3 tsh B–1
B.4 info B–2
B.5 fnnews and fnstocks B–2
B.6 fnbounce B–2
B.7 fnweather B–2
B.8 fnbattleship B–2
B.9 The Firmware B–3

About This Manual

This manual describes the software that turns a TFR/911H into a small, complete OS-9 system: NitrOS-9 Level 2, booted by the board's firmware, with a disk, a clock, a Unix-flavored set of commands, an editor, a sound chip, games, and a network connection through a FujiNet. It also describes the hardware that the firmware emulates for OS-9, and how to rebuild everything.

Audience

This manual is for anyone with a TFR/911H on their desk: people who know some Unix and want to find their way around OS-9, CoCo veterans who want to know what is different about this machine, and people who want to change the firmware or the disk. No knowledge of OS-9 is assumed in Chapter 1.

Organization

This manual has three chapters and two appendices:

Chapter 1 Introduces the TFR/911H, OS-9 Level 1 and Level 2, the TurbOS and NitrOS-9 systems that run on the board, and a first session.

Chapter 2 Describes networking: the FujiNet adapter, how it connects to the board, DriveWire, the FujiNet in NitrOS-9, and how to program it, with **fnwx**, the first **fnweather**, as the example, and five programs in C brought over from the Color Computer.

Chapter 3 Contains the reference pages, grouped as ULTRIX grouped them: Section 1, Commands; Section 4, Special Files (the hardware); Section 6, Games; and Section 8, Maintenance.

Appendix A Explains how to connect a speaker.

Appendix B Describes how the software was tested.

Related Documents

- The *TFR Installation Guide* (TFR-911H-IG), which tells how to install a release, flash the FujiNet, wire it to the board, and boot NitrOS-9 from it.
- `v4/nitros9-l2/README.md`, which this manual expands on.
- The NitrOS-9 documentation, at `nitros9.org` and in the NitrOS-9 source tree under `docs`.
- *OS-9 Level Two* for the Tandy Color Computer 3, whose commands NitrOS-9 still follows closely.
- The TFR/9 hardware reference, on the `strickyak/tfr9` wiki.

Conventions

Convention	Meaning
`OS9:`	The NitrOS-9 Shell prompt.
`TCL>`	The TFR/911H firmware's console prompt, before an OS boots.
`$`	A shell prompt on the computer the board is plugged into.
`bold type`	In examples, what you type.
italic	Something you supply: a file name, a number.
`[ ]`	In syntax, an optional part. Do not type the brackets.
|	In syntax, a choice: `on`|`off` means `on` or `off`.
Ctrl/C	Hold down the Ctrl key while you press C. Also written `^C`.
Return	The Return (Enter) key.
cmd(n)	The reference page for **cmd** in Section *n*.
`$FF41`	A hexadecimal number, often an address in the 6309's I/O page.

Introduction 1

This chapter introduces the TFR/911H, the two levels of OS-9, the two OS-9 systems the board runs, and what was added to NitrOS-9 Level 2 for it. It ends with a first session at the keyboard.

1.1 The TFR/911H

The TFR/911H is a single board computer built around a genuine Hitachi HD6309 microprocessor—the CMOS cousin of Motorola's 6809, with extra registers and a faster native mode—and a Raspberry Pi RP2350B microcontroller. Everything that is not the CPU is done by the RP2350B:

- It makes the 6309's E and Q clocks, one bus cycle at a time.
- It is the memory: 256K of the RP2350B's 520K of SRAM, behind an emulated MMU.
- It is every I/O device: a serial port for the console, a hard disk, a clock, interrupt timers, a sound chip.

One of the RP2350B's two cores runs the 6309's bus; the other talks USB to a program on your computer, the *tether*, which is the board's terminal, its disk drive, and its clock. The 6309 runs at about 2.8 to 3.2 MHz—faster than the 1.79 MHz of the fastest Color Computer.

The TFR/9 family of boards was designed to make very cheap 6809-family computers that run OS-9. Its design files and software are at `github.com/strickyak/tfr9`.

1.2 OS-9 Level 1 and Level 2

OS-9 was written by Microware Systems Corporation in 1979 and 1980 for the Motorola 6809. It is a real-time, multitasking, multi-user operating system that will look familiar to anyone who knows Unix: a hierarchical file system, a shell with pipes and redirection, devices that are files, processes with parents and children. It was written to be small and to run from ROM, so everything in OS-9 is a *module*: a position-independent, re-entrant block of code or data with a header and a CRC, which the system can find in memory by name. The kernel, the device drivers, and the commands you run are all modules.

The 6809 can address only 64K of memory, and OS-9 comes in two *levels* that differ in what they do about that.

Level 1 runs everything in a single 64K address space: the kernel, every process, every module. It needs no memory management hardware, which made it right for the first Color Computers. Tandy sold OS-9 Level One for the Color Computer in 1983.

Level 2 uses a memory management unit to give each process its own 64K address space, made of 8K blocks drawn from a larger physical memory—up to 2M. The kernel lives in a space of its own. Processes cannot see or damage each other's memory, and many more can run at once. Tandy sold OS-9 Level Two for the Color Computer 3, whose GIME chip has the MMU, in 1987.

NitrOS-9 began in the 1990s as a rewrite of OS-9 that made use of the Hitachi 6309's extra registers and instructions. Today the NitrOS-9 Project maintains it as open source, for both the 6309 and the 6809, for Level 1 and Level 2, and for many machines besides the Color Computer.

1.3 Level 1 on the TFR/911H: TurbOS

When the TFR/911H firmware starts, it waits at its console prompt, `TCL>`, a small Tcl interpreter (see **tcl**(8)). Typing **`bye`** there starts *TurbOS*, a small OS-9 Level 1 system that the firmware loads into the 6309's memory as a single 64K image, from its own flash.

TurbOS runs without a disk. Its kernel, I/O manager, SCF terminal manager, terminal driver, Shell, and a handful of commands (**mdir**, **mfree**, **procs**, **sleep**) are modules in the image. Its prompt is `TOS:`:

```
TOS:mdir
 Module directory at 00:00:59
kernel  init  tk  ioman  scf  scvt  term  go  shell  mfree  mdir  procs  sleep
```

TurbOS comes from the TFR/9 project's TurboLab work, in the `v4/turbolab` directory of the TFR/9 source. The rest of this manual is about Level 2.

1.4 Level 2 on the TFR/911H: NitrOS-9 V3.3.0

Typing **`l2`** at the `TCL>` prompt starts NitrOS-9/6809 Level 2 V3.3.0, built for the Color Computer 3. The firmware copies a small launcher and the kernel's boot track into memory and resets the 6309, which boots in about half a second:

```
NitrOS-9/6809 Level 2 V3.3.0
Tandy Color Computer 3
(C) 2014 The NitrOS-9 Project
Fri Oct 16 17:23:34 2026
http://www.nitros9.org

NitrOS-9 Level 2 on the TFR/911H.  Unix-style commands: man unix

Shell

OS9:
```

1.4.1 The machine NitrOS-9 sees

NitrOS-9 thinks it is on a Color Computer 3. The firmware emulates as much of one as OS-9 uses, and a little more:

Device	Address	What it is
RAM		256K in 32 blocks of 8K; 184K free for programs
GIME MMU	`$FF90`–`$FF91`, `$FFA0`–`$FFAF`	Two task maps; the constant page at `$FE00`
GIME timer	`$FF92`	The VBORD interrupt, 60 times a second: the system clock
6850 ACIA	`$FF14`–`$FF15`	The console, `/Term`
SN76489	`$FF41`	A sound chip: three tones and noise
Control	`$FF42`	Keyclicks, the bell, raw arrow keys
Info port	`$FF43`–`$FF44`	The firmware's report on itself
Becker port	`$FF45`–`$FF46`	DriveWire to a FujiNet: disks `/X0`–`/X3`, network `/N`
EmuDsk	`$FF80`–`$FF86`	A virtual hard disk, `/DD` and `/H0`
Clock	`$FFC0`–`$FFC7`	Date and time, from your computer

tfr911h(4) describes each of these.

1.4.2 The disk

The disk is a file on your computer, `v4/nitros9-l2/level2.dsk`: an RBF disk of 99,999 sectors (24.4M) that the tether serves to EmuDsk a sector at a time. It holds `OS9Boot`, the rest of the operating system, which the kernel

loads at boot; the commands in `/DD/CMDS`; the system files `errmsg` and `helpmsg` in `/DD/SYS`; and story files for **zm**(6) in `/DD/GAMES`. Whatever you save goes to the file at once.

With a FujiNet attached, typing **`l2fn`** instead of **`l2`** boots from the FujiNet's first device slot, a copy of this disk on its SD card or a TNFS server; `/DD` is then the DriveWire disk `/X0`, and the tether's disk is `/H0`. See **fujinet**(4).

1.4.3 The modules

These are in memory once the system is up (**info**(1) lists them):

Modules	What they do
`REL Boot Krn`	Relocate the boot track, load `OS9Boot` from EmuDsk (or, for **l2fn**, the FujiNet), and the kernel
`Init`	The system's settings: startup program, devices, table sizes
`KrnP2 KrnP3`	The rest of the kernel, and its error-message extension
`IOMan`	The I/O manager
`Clock Clock2`	The 60 Hz tick, and the real-time clock (`clock2_tfr911`)
`SCF sc6850 Term`	The terminal file manager, the 6850 driver, `/Term`
`RBF EmuDsk DD H0`	The disk file manager, the EmuDsk driver, `/DD` and `/H0`
`rbdw dwio X0–X3`	DriveWire disks on the FujiNet, and DriveWire's wire routines
`scdwv N N1–N5`	DriveWire's network devices, through which the **fn** commands talk
`PipeMan Piper Pipe`	Pipes
`SysGo Shell`	The first process, and the Shell it starts

1.5 What Was Added

These are the commands, games, and tools described in Chapter 3. Each has a **man** page on the board too: type **`man`** *name*.

Reference page	Summary
tsh(1)	The shell you type to: line editing, history, completion, and a search path
unix(1)	**ls**, **cd**, **cat**, **cp**, **mv**, **rm**, **ps**, **df**, and other Unix names for OS-9 commands
vi(1)	A small vi, also called **vim**
info(1)	Everything the system knows about itself
grep(1)	Print lines that contain a string
pwd(1)	Print the data or execution directory
clear(1)	Clear the screen
beep(1), **click**(1)	Ring the bell; turn keyclicks on and off
fujinet(1)	The FujiNet's hosts, device slots, and status: **fnstatus**, **fnlistdevs**, …
fnconfig(1)	FujiNet CONFIG
fnweather(1)	The weather, now and for the week
fnnews(1)	The news, from the FujiNet news server
fnstocks(1)	Ten stocks, with a ticker
fnbounce(1)	Bounce World: shapes that bounce across many computers' screens
fnbattleship(1)	Fuji Battleship: sea battle for up to four players
fujinet(4)	The FujiNet: its wiring, its disks, and booting from it
tfr911h(4)	The hardware the firmware emulates
sn76489(4)	Programming the sound chip
tetris(6)	Tetris, with music
zm(6)	A Z-machine, for Zork and the other Infocom games
tether(8)	The program that is the board's terminal and disk
tcl(8)	The firmware's console: **l2**, **l2fn**, **bye**, **fujinet**, **beep**, **click**
nitros9-l2(8)	Building the system and keeping a disk up to date

1.6 A First Session

Plug the TFR/911H into a USB port and, in the `v4` directory of the TFR/9 source, run `run-l2.sh`. It starts the tether, which boots NitrOS-9 by typing **`l2`** for you (or reconnects, if NitrOS-9 is already running):

```
$ WIRE=/dev/cu.usbmodem31201 sh run-l2.sh
TFR/911H on /dev/cu.usbmodem31201, disk .../nitros9-l2/level2.dsk.
^C goes to OS-9; quit with ^] .
```

Then look around. **man unix** lists the Unix-style names; **ls -l** lists a directory; **info** shows the whole system:

```
OS9:ls -l /dd/cmds
OS9:cd /dd
OS9:pwd
/DD
OS9:info
```

Write a file with **vi**, and look at it with **cat**:

```
OS9:vi notes
OS9:cat notes
```

Play something:

```
OS9:zm zork1
OS9:tetris
```

The shell is **tsh**: the arrow keys move about in the line you are typing and bring back the lines you typed before, and Tab finishes the name of a command or a file. **tsh**(1) has the rest.

Ctrl/C interrupts the program that is running, as on Unix. To leave the tether, press Ctrl/]; NitrOS-9 keeps running on the board, and `run-l2.sh` will reconnect to it.

OS-9 differs from Unix in a few ways you will meet early:

- Each process has two current directories: the *data* directory, for files (**cd**, **pwd**), and the *execution* directory, where commands are found (**chx**, **pxd**). Both start at `/DD` and `/DD/CMDS`.
- The pipe symbol is `!`, not `|`. `>>` redirects standard error; there is no append. `&` runs a command in the background, as on Unix.
- File names are not case sensitive. Text files end their lines with a carriage return, not a line feed.

Networking 2

This chapter describes how the TFR/911H reaches the network: the FujiNet adapter, how it is wired to the board and what the firmware makes of it, the DriveWire protocol that runs between them, how NitrOS-9 uses it, and how to write programs that use it. It ends with a walk through **fnwx**, the first **fnweather**, a program in assembly that finds where you are and shows the weather there, and with five programs in C, **fnnews**, **fnstocks**, **fnbounce**, **fnweather**, and **fnbattleship**, that came from the Color Computer with little change.

2.1 The FujiNet

A FujiNet is a network adapter for old computers. It began on the Atari 8-bit computers and now serves the Apple II, the Coleco ADAM, the Commodore machines, the Color Computer, and many more, each through its own peripheral bus. To the computer it is a set of devices—disk drives, network devices, a clock, a printer—and behind them it has WiFi and an SD card. A disk can be an image file on the SD card or on a server on the network; a network device can fetch a web page, pick fields out of the JSON in it, or hold a TCP connection open.

On the TFR/911H the FujiNet gives NitrOS-9:

- four disk drives, `/X0`–`/X3`, whose disks are image files the FujiNet reads from its SD card or a TNFS server;
- network devices, `/N` and `/N1`–`/N5`, through which programs ask the FujiNet to fetch from the network;
- a place to boot from: **l2fn** boots NitrOS-9 from the FujiNet's first disk;
- commands to manage it, **fujinet**(1) and **fnconfig**(1), and programs that use it: **fnweather**(1), **fnnews**(1), **fnstocks**(1), **fnbounce**(1), and **fnbattleship**(1).

2.1.1 The hardware

The FujiNet is an Espressif ESP32 on a small board. The one these pages were written with is an ESP32 development board with this module:

Part	What it is
CPU	ESP32-D0WD-V3: two Xtensa LX6 cores at 240 MHz
Memory	520K of SRAM, and 4M of PSRAM
Flash	8M: the firmware, and a LittleFS file system with the web pages
Radio	2.4 GHz WiFi 802.11 b/g/n (and Bluetooth, unused here)
SD card	For disk images, and the FujiNet's settings, `fnconfig.ini`
USB	A CP2102N USB serial bridge, for flashing and for the FujiNet's own log (460,800 baud)
Buttons	A, the DevKitC's BOOT button (GPIO 0)
LED	The bus activity light, GPIO 4

2.1.2 The firmware

The FujiNet runs *fujinet-firmware* (`github.com/FujiNetWIFI/fujinet-firmware`), one C++ code base built for each kind of computer. For the TFR/911H it is built for the board `fujinet-coco-devkitc-centipede`, which was added for the Centipede, a Color Computer cartridge with an RP2350 on it. That build selects:

`BUILD_COCO`	The Color Computer's devices.
`DRIVEWIRE`	DriveWire as the bus: the protocol a CoCo uses to reach a DriveWire server over a serial line.
`COCO_HS_UART`	A plain 3.3-volt serial link on the ESP32's UART1, at a fixed 921,600 baud, with no line inversion and no flow control—made for a microcontroller at the other end rather than a CoCo's bit-banger port.

Besides the DriveWire devices, the firmware joins your WiFi network, sets its clock by SNTP, and runs a web server. Its web pages show and change its settings, and its WebDAV server at `/dav` offers its SD card to your computer's file manager or to **curl**. Its settings—the WiFi network, the eight host slots, the device slots—are kept in `fnconfig.ini`, on the SD card and in flash.

The firmware must read the version byte that NitrOS-9 sends after `OP_DWINIT` (see "DriveWire" below); it does since pull request 2027 to fujinet-firmware, merged in October 2026. On a DevKitC it must also have no Safe Reset button. The board's pin map gave the button GPIO 14, which a DevKitC leaves unconnected, and the firmware sets no pull-up on it, so the pin floats, picks up the UART beside it, and reads as presses; each one restarts the FujiNet, and under heavy disk traffic, as when NitrOS-9 boots from it, that is every few seconds. The fix, which takes the button out of the DevKitC's pin map, is pull request 2034, merged in October 2026 as well, so firmware built from fujinet-firmware since then has both fixes. Release jamesBrown1 has such firmware, built for FujiNet-Flasher; the *TFR Installation Guide* tells how to flash it. `v4/nitros9-l2/fujinet/README.md` tells how to build and flash the firmware.

2.2 How the FujiNet Connects to the TFR/911H

2.2.1 The wiring

The FujiNet needs three wires to the TFR/911H, wired as the Centipede's FujiNet hat is:

RP2350B	ESP32	Signal
GPIO 12 (UART0 TX)	GPIO 33 (UART1 RX)	Data to the FujiNet
GPIO 13 (UART0 RX)	GPIO 21 (UART1 TX)	Data from the FujiNet
GND	GND	

Both ends are 3.3-volt parts, so they connect directly. The link runs at 921,600 baud, 8 data bits, no parity, one stop bit, with no flow control: about 92,000 bytes a second, or a 256-byte sector in under 3 ms. The FujiNet's own USB port can stay plugged into your computer, which powers it and shows its log.

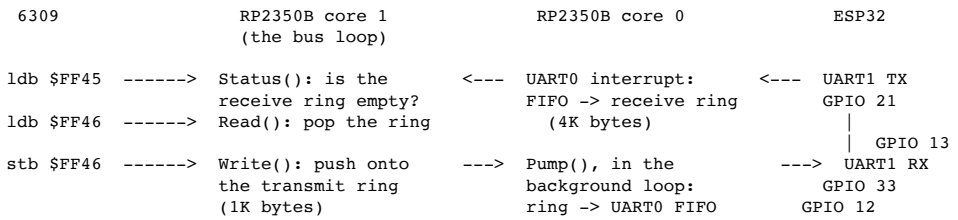
2.2.2 Inside the RP2350B

The 6309 cannot see a serial port; it sees only what the RP2350B answers on its bus. The firmware puts the link in the 6309's I/O page as a *becker port*, the simple two-register interface that CoCo emulators give DriveWire (`v4_fujinet.h`, in `v4/tfr911h`):

`$FF45`	Status. Bit 1 is set when a byte from the FujiNet is waiting.
`$FF46`	Data. Reading takes the next byte from the FujiNet; writing sends a byte to it.

Emulators put the becker port at `$FF41`, but that is where the TFR/911H has its sound chip, so the port is four bytes higher, and NitrOS-9's DriveWire modules are built to match.

The RP2350B's two cores share the work:



Core 1 answers every bus cycle of the 6309, with interrupts off, and must never wait. So it touches only two rings in memory, never the UART. The UART's own buffers hold only 32 bytes, which at 921,600 baud is about 350 mi-

croseconds; core 0's background loop can be busy longer than that, sending the console or a disk sector to the tether over USB. So the UART's receive interrupt, on core 0, empties the UART into a 4K receive ring as bytes arrive, and nothing is lost. Sending has no such hurry, and the background loop moves bytes from the transmit ring to the UART when it comes round.

Reading `$FF46` when nothing is waiting gives `$FF`, and counts an "empty read"; a correct program checks `$FF45` first.

2.2.3 Watching the link

At the `TCL>` prompt, before an OS boots:

```
TCL> fujinet
FujiNet says it is 2026-10-17 18:46:20
TCL> fujinet stats
rx 6 tx 1 rx_drops 0 tx_drops 0 overruns 0 empty_reads 0
```

fujinet sends the DriveWire `OP_TIME` request and prints the FujiNet's answer, which proves both wires and the baud rate. While NitrOS-9 runs, the firmware's statistics in the tether's log, `v4/_log`, include a line every ten seconds with the byte counts and the last 32 bytes on the wire, > for bytes to the FujiNet and < for bytes from it:

```
FujiNet: 12666 out, 97015 in, 0 lost, 0 empty reads; last: >e3 >01 >52 >00 >07 >e3 ...
```

info(1) shows the same counts in its hardware report.

2.2.4 Booting from the FujiNet

The firmware holds two copies of NitrOS-9's boot track. **l2** starts the one whose `Boot` module reads `OS9Boot` from EmuDsk, the tether's disk. **l2fn** starts the one whose `Boot` module, `boot_dw`, reads it from DriveWire drive 0—the FujiNet's first device slot. When the FujiNet powers up, it puts its own CONFIG disk in that slot, so **l2fn** first mounts whatever the slot names, with the FujiNet's own commands (`FUJI$MountHost`, then `FUJI$MountImage`; see "Programming the FujiNet"):

```
TCL> l2fn
FujiNet slot 1: level2-fn.dsk
...
NitrOS-9 Level 2 on the TFR/911H
```

The `OS9Boot` on that disk makes `/X0` the system disk, `/DD`. The tether's disk is still there, as `/H0`. **fujinet**(4) tells how to make such a disk and put it on the FujiNet.

2.3 DriveWire

DriveWire, from Cloud-9, is the protocol a Color Computer uses to reach a DriveWire server on another computer: a serial line, and *transactions* that each begin with a one-byte opcode from the CoCo. The CoCo always asks and the server answers; the server never speaks first. These are the transactions the TFR/911H uses:

Opcode	Name	What it does
$D2	OP_READEX	Read a sector: drive and 24-bit LSN out; 256 bytes back; a checksum out; a status byte back
$F2	OP_REREADEX	The same, after a checksum error
$57	OP_WRITE	Write a sector: drive, LSN, 256 bytes and a checksum out; a status byte back
$23	OP_TIME	Six bytes back: year since 1900, month, day, hour, minute, second
$5A	OP_DWINIT	The driver's version byte out; the server's version byte back
$45	OP_SERINIT	Open a virtual channel (/N1 is channel 1)
$C4	OP_SERSETSTAT	A SetStat on a virtual channel: SS.Open, SS.ComSt (with 26 bytes of path options), …
$E2	OP_FUJI	A FujiNet management command (below)
$E3	OP_FUJINET	A FujiNet network-device command (below)

Reading a sector looks like this on the wire (the numbers are an example). The checksum is the sum of the 256 bytes, as a 16-bit number; if the server's sum differs, it answers E$CRC and the driver asks again with OP_REREADEX:

```
6309 -> FujiNet:   D2  00  00 00 3C             OP_READEX, drive 0, LSN $00003C
FujiNet -> 6309:   ... 256 bytes of sector $3C ...
6309 -> FujiNet:   7A 41                        the checksum it computed
FujiNet -> 6309:   00                           status: no error
```

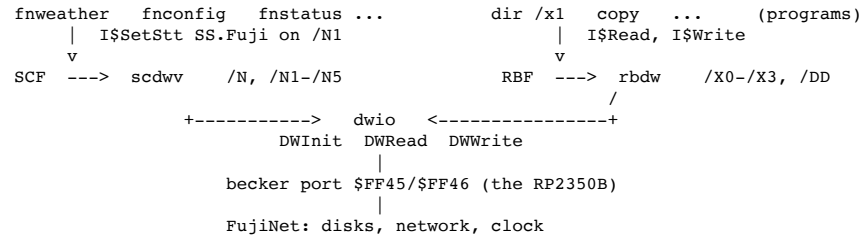
The FujiNet adds two opcodes of its own, $E2 and $E3, which are how programs reach its management functions and its network devices. Their replies come back as raw bytes, outside DriveWire's virtual-channel framing.

A DriveWire link has no framing or resynchronization: both ends must agree on the length of every transaction. A server that reads one byte too few takes the next byte for an opcode, and both ends are lost. That is what happened before the FujiNet's firmware was fixed to read the version byte after OP_DWINIT: it took that byte for an opcode, threw away the OP_SERINIT and OP_SERSETSTAT already waiting behind it, and the 6309 waited for a reply that never came. On a real CoCo's slower link the next command had not yet arrived, and survived.

2.4 The FujiNet in NitrOS-9

2.4.1 The modules

NitrOS-9 has had DriveWire drivers for years. The FujiNet uses them, and adds a way for programs to send the FujiNet's own commands:



Module	What it does
dwio	DriveWire's wire routines: DWInit, DWRead, and DWWrite, reached through the system variable D.DWSubAddr. Built with BECKER=1 and BECKBASE=$FF45, so they poll the becker port; and with DWIO_NO_VIRQ=1 (see below).
rbdw	The RBF driver for DriveWire disks.
X0–X3	Its descriptors, drives 0–3 (the FujiNet's device slots 1–4). DD, booting from the FujiNet, is drive 0 too.
scdwv	The SCF driver for DriveWire's virtual channels, with the SS.Fuji SetStat for FujiNet commands.
N, N1–N5	Its descriptors: channels 1–5, and /N, the wildcard. The FujiNet commands use /N1.
boot_dw	For **l2fn**: the Boot module that loads OS9Boot from drive 0, with its own copy of the wire routines.

The source of scdwv and dwio, the FujiNet's definitions in drivewire.d, the libraries, and the commands are in v4/nitros9-l2/fujinet; the rest come from the NitrOS-9 tree.

2.4.2 Synchronous, with interrupts masked

Every DriveWire exchange in NitrOS-9 is synchronous: the driver masks the 6309's interrupts, writes its request, and polls for each byte of the reply. Nothing else can use the wire meanwhile, so no one else's bytes are taken for yours. The cost is that the clock and the keyboard wait too: a request that keeps the FujiNet busy for a second stops NitrOS-9 for a second.

DriveWire's *virtual-serial poller* breaks this rule. In a normal DriveWire system dwio installs a routine on the 60 Hz clock that asks the server, between other work, whether a virtual channel has data (OP_SERREAD). The FujiNet's replies are raw bytes, not channel data, and a poll between a request and its reply would take them. So dwio is built with DWIO_NO_VIRQ=1, which leaves the poller out. /N1 still opens and closes, but programs talk through it only with SS.Fuji.

2.4.3 Level 1 and Level 2

Like most of NitrOS-9, the FujiNet code is written once for both levels: its sources live in the Level 1 directories of the NitrOS-9 tree, and conditional assembly on Level picks what differs. The commands and libraries are the same source for both, linked with the system definitions for one level or the other. The drivers differ in how they reach memory:

Level 1 has one 64K address space. scdwv's SS.Fuji hands the caller's request buffer straight to DWWrite, and has DWRead put the reply straight into the caller's reply buffer.

Level 2 gives each process its own address space, which the driver, running in the system's address space, cannot address. So SS.Fuji takes a 512-byte buffer from system memory (F$SRqMem), copies the request into it from the caller's task with F$Move, makes the exchange from that buffer, copies the reply back to the caller's task with F$Move, and returns the buffer (F$SRtMem).

On the TFR/911H, Level 1 is TurbOS, a 64K image in the firmware with no disk or DriveWire modules, so the FujiNet is a Level 2 feature here. The same Level 1 code runs elsewhere: the dillera/nitros9 tree, where it was written, has a Level 1 recipe that boots a Wildbits Jr2 from a FujiNet over its UART.

2.5 Programming the FujiNet

A program reaches the FujiNet by opening /N1 and making SS.Fuji SetStat calls on the path. Each call is one complete exchange: a request sent, and a reply of a known length read back. Two libraries build the requests, one for the FujiNet's management commands and one for its network devices. They are in fujinet/lib, in v4/nitros9-l2.

2.5.1 The SS.Fuji call

A	The path to /N1.
B	SS.Fuji ($E2).
X	A parameter block: the length of the reply wanted (two bytes, high byte first), then the request.
Y	The length of the parameter block: 2 plus the request's length.
U	Where the reply goes. Not used if the reply length is 0.

The request and the reply may each be up to 512 bytes (FUJI$MaxTran). On success the carry is clear. Because a program keeps its data pointer in U, save it around the call, as the libraries do:

```
* Ask the FujiNet for the status of the last management command: send
* [OP_FUJI, FUJI$GetError] and read one byte back into fbstat.
FBErr               pshs      a,x,y,u
                    leax      fbsmall,u
                    ldd       #1                  one byte of reply wanted
                    std       ,x
                    ldd       #OP_FUJI*256+FUJI$GetError
                    std       2,x                 the request: two bytes
                    ldy       #4                  2 + the request
                    lda       fbpath,u            the path to /N1
                    ldb       #SS.Fuji
                    pshs      u
                    leau      fbstat,u            the reply goes here
                    os9       I$SetStt
                    puls      u
```

Ask for exactly as many bytes as the command sends back. If you ask for more, the 6309 waits for bytes that will never come—with interrupts masked—and only a reset ends the wait.

2.5.2 Management commands

A management command is OP_FUJI ($E2), a command byte, and its parameters. The FujiNet carries it out and keeps the outcome; the program then asks for the outcome with FUJI$GetError, which returns $01 if it worked, and for any data with FUJI$GetResponse. lib/fuji.as does this:

NOpen	Open /N1; keep the path.
FBReady	Ask until the FujiNet says it is ready (FUJI$Ready).
FBCmd	Send a command, then fetch its outcome; carry set if it failed.
FBErr	Fetch the outcome of the last command.
FBRead	Fetch the data of the last command.

The commands are defined in fujinet/defs/drivewire.d:

Byte	Name	What it does
$E8	FUJI$GetAdapterCfg	The adapter's settings: SSID, host name, IP, gateway, netmask, DNS, MAC, firmware version (140 bytes)
$FA	FUJI$GetWifiStatus	Whether the WiFi is connected
$FD, $FC	FUJI$ScanNetworks, GetScanResult	Scan for WiFi networks; one result
$FE, $FB	FUJI$GetSSID, SetSSID	The WiFi network and password
$F4, $F3	FUJI$ReadHostSlots, WriteHostSlots	The eight host slots, 32 bytes each
$F9, $E6	FUJI$MountHost, UnmountHost	Connect to a host, or stop
$F7, $F6, $F5	FUJI$OpenDir, ReadDirEntry, CloseDir	List a directory on a host
$F2, $F1	FUJI$ReadDevSlots, WriteDevSlots	The device slots: host slot, mode, and a 36-byte file name each
$E2, $DA	FUJI$SetDevicePath, GetDevicePath	One device slot's image
$F8, $E9	FUJI$MountImage, UnmountImage	Mount or eject a device slot's image
$E7	FUJI$NewDisk	Make a new, empty disk image
$02, $01	FUJI$GetError, GetResponse	The outcome, and the data, of the last command
$00	FUJI$Ready	One byte back, nonzero when the FujiNet is ready

All of **fnstatus** is a page of code. After opening `/N1`, it asks for the adapter's settings and prints them:

```
__start             lbsr    NOpen               open /N1
                    lbcs    errex
                    sta     netpath,u
                    lbsr    FBReady             wait until the FujiNet answers
                    lbcs    closeerr
                    leax    request,u
                    ldd     #OP_FUJI*256+FUJI$GetAdapterCfg
                    std     ,x
                    ldy     #2
                    lbsr    FBCmd               send it; check the outcome
                    lbcs    closeerr
                    leax    config,u
                    ldy     #ADPCFGSZ           140 bytes of settings
                    lbsr    FBRead
                    lbcs    closeerr
                    ...                         print config+CFGVER, config+CFGHOST, ...
```

2.5.3 The network device

The FujiNet's network devices (its `N:` devices) fetch from the network for the computer. A network command is `OP_FUJINET` (`$E3`), a unit number (`fnnet` uses unit 1), a command byte, two parameter bytes, and, for some commands, a 256-byte *device spec*, padded with zeros:

```
E3  01  'O'  04  00   N:https://api.open-meteo.com/v1/forecast?latitude=...  00 00 ...
|   |    |   |   |    '--------------- device spec, 256 bytes -------------------'
|   |    |   |   '-- translation: 0, none
|   |    |   '------ mode: 4, read
|   |    '---------- command: open
|   '--------------- unit 1
'------------------- OP_FUJINET
```

Command	Parameters	What it does
`'O'`	mode, translation; spec	Open the URL in the spec. Mode 4 is read, for HTTP GET
`$FC`	1, 0	Channel mode: 1 makes the device parse what it fetched as JSON
`'P'`		Parse: fetch the URL, and parse the JSON
`'Q'`	; spec	Query: the spec is a path into the JSON, such as `/current/temperature_2m`
`'R'`	length (2)	Read up to *length* bytes of the open device—after `'Q'`, the value
`'S'`		Status: bytes waiting, connected, and the last error
`'C'`		Close
`$02`		The outcome of the last command, one byte: `$01` if it worked
`$01`	length (2)	The data of the last read, padded with zeros to *length*

The device does more than this—writing, and other protocols in the spec, such as TCP, UDP, FTP, TNFS, and SSH—but HTTP and JSON are what `lib/fnnet.as` provides:

`FNNetInit`	Open `/N1`, and give the FujiNet a moment.
`FNNetOpen`	Open the URL at `X` (up to 255 characters, beginning `N:`).
`FNNetParse`	Select JSON mode and parse what the URL returns, then wait ten seconds, since an HTTPS request can take the ESP32 several, before the first query.
`FNNetQuery`	Look up the JSON path at `X`; put the value, as text, at `Y`, at most `B` bytes with its terminating zero.
`FNNetClose`	Close the device.

So fetching one value from a web service is four calls:

```
                    lbsr      FNNetInit           once, at the start: open /N1
                    leax      url,pcr
                    lbsr      FNNetOpen
                    lbsr      FNNetParse
                    leax      query,pcr           "/city"
                    leay      city,u              33 bytes, in the program's data
                    ldb       #33
                    lbsr      FNNetQuery          city now holds "Philadelphia"
                    lbsr      FNNetClose
   ...
url                 fcn       "N:http://ip-api.com/json/?fields=status,city"
query               fcn       "/city"
```

(Check the carry after each call; the real programs do.)

2.5.4 Building a program

The FujiNet programs are LWASM object files, linked with LWLINK into OS-9 modules. A program has three sections: `__os9`, with its module type, edition and stack size; `bss`, its variables, which the linker puts in the process's data area, reached through `U`; and `code`. Execution starts at `__start`. The build links each one with the libraries:

```
$ lwasm --6309 --format=obj -DANSI=1 -Ifujinet/lib -Ifujinet/defs -I$(NITROS9_DIR)/defs \
      -o build/fn/fnhello.o fujinet/cmds/fnhello.as
$ cd build/fn && lwlink --format=os9 -o fnhello fnhello.o fnnet.o fuji.o ../sys.o ../alib.a
```

The Makefile in `v4/nitros9-l2` does this for every file in `fujinet/cmds`: put `fnhello.as` there, type **`make`**, and **fnhello** is in `/DD/CMDS` on the new disk (**`make install`** puts it on yours). `alib` gives you `PRINTS`, `PUTS`, `DEC_BIN`, and NitrOS-9's other assembly-language helpers.

2.6 An Example: fnwx

fnwx (`fujinet/cmds/fnwx.as`, 1,213 lines) finds where you are from your network's public address, and shows the weather there. It was the first **fnweather**, until that was written again in C, with icons and the week's forecast (see Programs in C); it is kept, as **fnwx**, for this walk through. It uses two free web services, neither of which needs a key: `ip-api.com` for the location, and Open-Meteo for the weather.

2.6.1 The plan

1. Put the terminal in raw mode, and open `/N1` (`FNNetInit`).
2. *Locate*: fetch `ip-api.com`, and query the city, region, country, latitude, and longitude.
3. Choose Fahrenheit and miles in the United States, Liberia, and Myanmar, Celsius and kilometres elsewhere.
4. *Fetch*: build an Open-Meteo URL from the latitude and longitude, and query the conditions—twice, since one URL with every field would not fit in the 256-byte device spec.
5. Draw the screen, and wait for a key: R fetches again, U changes the units, L locates again, Q quits.

2.6.2 Locating

The location request names only the fields it wants, so the reply is short:

```
ipURL     fcn   "N:http://ip-api.com/json/?fields=status,city,regionName,countryCode,lon,lat"
```

and the answer is a small JSON object, for example:

```
{"status":"success","countryCode":"US","regionName":"Pennsylvania",
 "city":"Philadelphia","lat":39.98,"lon":-75.23}
```

`FetchLocation` opens it, parses it, checks that `/status` is `success`, and copies each field into a buffer of its own:

```
FetchLocation       ...
                    leax      ipURL,pcr
                    lbsr      FNNetOpen
                    lbcs      FetchAbort
                    lbsr      FNNetParse
                    lbcs      FetchAbort
                    leax      qStatus,pcr         "/status"
                    leay      linebuf,u
                    ldb       #16
                    lbsr      FNNetQuery
                    lbcs      FetchAbort
                    leax      linebuf,u
                    leay      success,pcr         "success"
                    lbsr      StrEqual
                    bne       FetchLocationBad
                    leax      qCity,pcr           "/city"
                    leay      city,u
                    ldb       #33
                    lbsr      FNNetQuery
                    ...                           /regionName, /countryCode, /lat, /lon
                    lbsr      FNNetClose
                    rts
```

The values arrive as text, just as they appear in the JSON: the latitude is the string `39.98`, ready to be put into the next URL.

2.6.3 Fetching the weather

`BuildWeatherURL` joins the pieces in a 256-byte buffer:

```
meteoHead      fcn  "N:https://api.open-meteo.com/v1/forecast?latitude="
meteoLongitude fcn  "&longitude="
currentTail1   fcn  "&timezone=auto&forecast_days=1&current=temperature_2m,
                     apparent_temperature,relative_humidity_2m,weather_code"
imperialTemp   fcn  "&temperature_unit=fahrenheit"

BuildWeatherURL     pshs      x                   X = the tail to end with
                    leay      urlbuf,u
                    leax      meteoHead,pcr
                    lbsr      CopyZAdvance        copy, leaving Y at the end
                    leax      latitude,u
                    lbsr      CopyZAdvance
                    leax      meteoLongitude,pcr
                    lbsr      CopyZAdvance
                    leax      longitude,u
                    lbsr      CopyZAdvance
                    puls      x
                    lbsr      CopyZAdvance
                    clr       ,y
                    rts
```

then `FetchCurrentOne` opens it, parses it, and queries five values under `/current`: the time, `temperature_2m`, `apparent_temperature`, `relative_humidity_2m`, and `weather_code`. A second URL asks for the pressure and the wind.

Here is one of those queries on the wire, from the TFR/911H's log: the wind direction. After the query (`'Q'`, `$51`, with its path), the program reads up to seven bytes of the value, fetches them, closes the device, and checks the outcome:

```
>e3 >01 >52 >00 >07                 'R': read, up to 7 bytes
>e3 >01 >01 >00 >07                 $01: the data, 7 bytes
<33 <34 <31 <0a <00 <00 <00         "341", a line feed, and padding
>e3 >01 >43 >00 >00                 'C': close
>e3 >01 >02 >00 >00                 $02: the outcome?
<01                                 $01: it worked
```

`FNNetQuery` copies the value up to the line feed, so the program has the string `341`, which `WindDirection` turns into a compass point with `DEC_BIN` and a few comparisons: 341 degrees is north.

2.6.4 Turning numbers into words

Open-Meteo gives the sky as a WMO weather code. `WeatherDescription` turns it into words the same way:

```
WeatherDescription  lbsr      DEC_BIN            the code, as text, into D
                    bcs       DescUnknown
                    cmpd      #0
                    beq       DescClear          "Clear sky"
                    cmpd      #1
                    beq       DescMainly         "Mainly clear"
                    ...
                    cmpd      #58
                    blo       DescDrizzle        51-57: "Drizzle"
                    ...
```

2.6.5 The screen

fnwx reads keys one at a time, without echo: it copies its standard input's path options with `SS.Opt`, turns off echo, auto line feed, and paging (`PD.EKO`, `PD.ALF`, `PD.PAU`), and puts them back when it ends. It draws with a handful of routines: clear the screen, move the cursor (`TAt`, column in `A` and row in `B`), cursor on and off, and reverse video. For a Color Computer 3 window they send its control codes; built with `ANSI=1`, as on the TFR/911H, they come from `fujinet/lib/fnansi.as` and send ANSI sequences to the tether's terminal instead. The rest of the program does not know the difference.

2.6.6 Writing your own

Any web service that answers a GET request with JSON can be used the same way: build the URL, open it, parse it, query the fields you want, close it. Some things to remember:

- A device spec, with its `N:` and its URL, is at most 255 characters. Ask the service for only the fields you need, as **fnwx** does; split the request in two if you must.
- A query's value is text, as it appears in the JSON. Give `FNNetQuery` a buffer big enough for the longest value, with its zero.
- An array element is a path too: `/daily/time/0` is the first element of `daily.time`.
- Each exchange stops NitrOS-9 while it runs, and each `FNNetParse` takes ten seconds. Fetch when the user asks, not in a loop.
- Close what you open, even when a query fails, as `FetchAbort` does; a device left open keeps the FujiNet's connection, and the next program's replies may not be what it expects.

fnconfig is the other large example: the management commands for hosts, device slots, directories, and WiFi, behind a full-screen interface.

2.7 Programs in C

Many of the FujiNet's programs for other computers are written in C, against *fujinet-lib*, the FujiNet's C library, which has the same calls on every computer. Those for the Color Computer are compiled with CMOC, which compiles for OS-9 too. `fujinet/fujinet-lib`, in `v4/nitros9-l2`, is fujinet-lib for NitrOS-9, and `fujinet/c` is a terminal to draw on. With them, five of the Color Computer's programs became NitrOS-9 commands, their logic unchanged: **fnnews**(1), from `fujinet-news`, **fnstocks**(1), from `fujinet-stocks`, **fnbounce**(1), from `bounce-world-client`, **fnweather**(1), from `fujinet-weather`, and **fnbattleship**(1), from `fujinet-battleship`.

2.7.1 fujinet-lib for NitrOS-9

fujinet-lib keeps what is the same on every computer in `common`, and what each computer's bus needs in `bus/`*platform*. On a CoCo the bus calls the DriveWire ROM: `dwwrite` sends bytes and `dwread` reads the reply, and the rest of the CoCo's bus—`fuji_bus_call`, `bus_ready`, the calls for a command's status and its data—is built on those two. `fujinet/fujinet-lib` has fujinet-lib v5.0.1's `include` and `common`, unchanged, and a bus for NitrOS-9, `bus/os9`, made from the CoCo's:

`ssfuji.c`	Opens `/N1`, and makes the `SS.Fuji` calls.
`dwwrite`	Queues its bytes.
`dwread`	Sends what is queued and reads the reply, in one `SS.Fuji` call. Every fujinet-lib command ends with a read, of its status at least, so nothing is left queued.
`network_bus.c`	Reads at most 512 bytes at a time, as much as `SS.Fuji` carries; `network_read` asks again for the rest.
`fuji_bus_call.s`	The CoCo's, made position independent, as an OS-9 module must be: `LBSR` for its calls, its variables through `Y`, and its table through `PCR`.
`network_proceed.c`	A becker port has no proceed line: poll `network_status` instead.

The rest of the bus is the CoCo's, unchanged, and so is the protocol on the wire, since the FujiNet runs the same firmware. Every `fuji_*` and `network_*` call of fujinet-lib is there—HTTP and JSON, TCP and UDP, app keys (up to 64 bytes the FujiNet keeps on its SD card for a program), host and device slots, directories, the clock—and a program written with fujinet-lib for the CoCo builds for NitrOS-9. `README.md` in the directory tells how to bring in a newer fujinet-lib.

A call is a few `SS.Fuji` exchanges. Each command is preceded by `bus_ready` (`$E2 $00`, which the FujiNet answers with one byte) and followed by `$02`, its outcome; `network_open`, for instance, sends `'O'` with its spec. fujinet-lib asks for the outcome of a JSON parse (`'P'`) at once, where `fnnet.as` sleeps ten seconds first. The FujiNet does not read the question until it has fetched and parsed the document, and the becker port has no timeout, so the call takes as long as the fetch and no longer. NitrOS-9 stops while it waits.

These are the four calls of the last section, in C:

```
#include "fujinet-network.h"

char city[64];
char url[] = "N:http://ip-api.com/json/?fields=status,city";

if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) == FN_ERR_OK
    && network_json_parse(url) == FN_ERR_OK)
    network_json_query(url, "/city", city);       /* "Philadelphia" */
network_close(url);
```

2.7.2 The terminal

The Color Computer's programs draw on its screen: `printf` at the cursor, and reverse video for the line that is selected, made by flipping the attribute bytes of a row in screen memory. `term.c` draws the same things on the tether's terminal with ANSI sequences. It keeps a copy of the screen, a byte for each place, with bit 7 set where it is in reverse video, and a byte of its colors, so that `term_invert_row` can flip a row and send it again; `term_color` sets the colors of what follows, from the terminal's eight and their bright versions. It reads keys raw from `/term`, sets bit 7 of `$FF42`, as **vi**(1) does, so that the arrow keys arrive as the tether sends them, and takes Esc, and Ctrl/C, which arrives as a signal, as the Color Computer's BREAK.

A program that redraws the screen many times a second, as a CoCo program does by drawing on a second screen and then showing it, calls `term_frame` to start a frame, which is drawn in the copy alone, and again to end it, which sends only the places that changed; only the rows the frame wrote are compared. A place can hold one of the line-drawing and block characters (`TG_HLINE` and the rest), which `term.c` sends as UTF-8. And what it sends goes straight to the console's 6850, at `$FF15`, not through SCF, which takes some 300 microseconds a character: the firmware's 6850 is always ready for another character, and holds the 6309 while its buffer to the tether is full.

2.7.3 The five programs

fnnews is the Color Computer client of `fujinet-news`. Its topics, pages of headlines, and pages of a story are the same state machines as on the CoCo, laid out as on a CoCo 3's 80-column screen. The news server formats each page to fit; the program puts the screen's width and height in the URL:

```
N:HTTPS://FUJINET.ONLINE/8bitnews/news.php?t=lf&ps=80x21&l=6&p=1&c=top
```

The CoCo's screen code, `cocotext.c` and `bar.c`, became `fujinet/news/screen.c`, a page of calls to `term.c`, which also draws the title bars, rules, and colors, and the page `I` shows, about the program and the server.

fnstocks is `fujinet-stocks`, written for many computers at once: its own code calls the `conio` routines and the C library, and each computer has a directory of headers and routines that provide them. NitrOS-9 has one, `os9`, beside `coco`. Its `conio.h` sends `gotoxy`, `revers`, `cputs`, and the rest to `term.c`; its `time.h` makes `clock` from `F$Time`. Of the program's own files, a few lines changed: an 80-column screen, `Esc` as the name of `BREAK`, the ticker drawn only when it moves, since on a terminal each row drawn goes down the wire, and colors, through a `COLOR` macro that is empty on other computers. The stock list is kept in two app keys of creator `$0901`, the ones the other computers' versions use, so a FujiNet shared between computers has one list.

fnbounce is the client of Bounce World, Mark Fisher's `bounce-world-client`, which is written for many computers at once too: NitrOS-9's directory, `os9`, is beside `coco`. A server moves shapes about a world made of the screens of all its clients, and sends each client, ten times a second over a TCP connection, the shapes on its screen. The CoCo draws them on a 42-column screen. Here each place of the world is two columns of the terminal, so that a round shape looks round; the line-drawing characters keep their vertical strokes in the left column and carry their horizontal ones through the right, so that the lines still join; or, set on its title screen, the world is 80 places wide, a place to a column. The screens before the world—the title, with its logo and settings, and the kinds of shape—are NitrOS-9's own (`os9/title.c`), drawn for an 80-column terminal. Of the program's own files, a few lines changed (`BUILD_OS9`): the places and a color for each kind of shape, calls to those screens, and the main loop, which sends the next request before it draws, so that the FujiNet and the server are at work while the 6309 draws.

fnweather is the Color Computer client of `fujinet-weather`: its requests to `ip-api.com` and Open-Meteo, its descriptions of the WMO weather codes and its choice of icon for each, with a screen of its own for the terminal (`fujinet/weather`). It asks for the weather in three requests, not the CoCo's seven, the fewest whose URLs fit a device spec of 255 characters, and in metric units, which it converts itself, in integers, since CMOC has no floating point under OS-9; so `U` needs no request. Its icons are pixel art, two pixels to a character, drawn with half blocks in the terminal's colors, one as the foreground and one as the background where both halves are colored; a Python program, `icons.py`, makes them from shapes—a sun, clouds of overlapping discs, drops, flakes, a bolt—into `icondata.h`. What it fetched last is kept in `/DD/SYS/fnweather.dat`, with the place and the units, so that it shows them at once, and fetches again only when they are half an hour old.

fnbattleship is `fujinet-battleship`, Eric Carr's and Thomas Cherryhomes' game of sea battle, whose game is the same C on every computer, with a directory for each computer's screen, keys, and sound; NitrOS-9's, `os9`, is beside `coco`. The server sends its state as a C structure, ready to read into memory, and CMOC lays structures out as the server expects, a byte at a time with no padding. On the CoCo the boards are bitmaps; here a square is two columns of the terminal, so that it is square, the water blue with waves, the ships in grey half blocks, hits a fire of shaded blocks in red and yellow, misses a white ring, and the cursor a blinking reticle. Two players have their boards side by side, with coordinates, and whose turn it is between them; three or four have a quarter of the screen each. A coordinate can be typed, a letter and a digit, as well as aimed at with the arrows, and the ship being placed is marked in the fleet beside the board (`BUILD_OS9` in `gamelogic.c`). One change is for every computer: when a turn's time runs out, the program asks the server for the state at once. Upstream's waited out the time again, and again, while the server had moved for the player. Its sounds, played on the SN76489, are a sonar's ping for your turn, a shell's whistle, explosions—noise on the noise voice at the pitch of a silent tone voice, swept down—and splashes. `term.c` keeps 31 graphics characters at

most, and lets a program have its own (`term_glyphs`); **fnbattleship** has the double lines, shades, arrows, and rings it draws with. Its title has a battleship of pixel art, made by `shipart.py` as **fnweather**'s icons are.

2.7.4 Building and testing

`make fujinet-lib` compiles fujinet-lib and puts it in `build/fnlib/libfujinet.a`. The Makefile compiles each program's C files with `cmoc --os9 -c`, and links them with `term.o` and the library, from which only what the program uses is taken:

```
$ cmoc --os9 --cpp=$PWD/fujinet/fujinet-lib/cmoc-cpp -Ifujinet/fujinet-lib/include -Ifujinet/c \
      --function-stack=0 --add-os9-stack-space=2048 -DBUILD_OS9 -c -o build/fn/news/main.o \
      fujinet/news/main.c
$ cd build/fn && cmoc --os9 -o fnnews news/*.o c/term.o -L../fnlib -lfujinet
```

fujinet-lib indents some of its preprocessor directives, which macOS's **cpp**, a traditional preprocessor, skips; `cmoc-cpp` gives CMOC the C compiler's own preprocessor instead. The stack space is set when the file with `main` is compiled; given when linking, it is ignored, and the program has no room for its stack. CMOC's OS-9 programs keep their stack above their variables, in the same data area. And `term_printf` does its own formatting, because CMOC's `vsprintf` does not work in an OS-9 program (in version 0.1.97 it writes to standard output, and the stack then overflows); `printf` and `sprintf` do. Two more of CMOC's ways showed on the board, in **fnweather**. A `const` array of strings, such as `const char t[10][14][15]`, is kept in the data area with the variables that have values to start with, where a `const char t[]` is kept in the module; and the module's header, which tells OS-9 how much memory the program needs, counts the variables that start at zero and the stack, but not those. A big table of strings therefore leaves the stack no room, and **fnweather**'s icons are each one long string. And the product of an `unsigned char` and a constant that fits in one is kept to eight bits: `icon * 196` has to be written `(word) icon * 196`. **`make build/fnnews-host`** and **`make build/fnstocks-host`** build them for the computer the tether runs on, with `fujinet/c/host/fujinet-host.c` in place of fujinet-lib: **curl** fetches, **python3** answers the JSON queries, and app keys are files. The screens are the same, so a change can be tried there before it goes to the board. **`make build/fnweather-host`** builds **fnweather**, which keeps its cache in `fnweather.dat` there, or in `$FNWEATHER_CACHE`. **`make build/fnbattleship-host`** builds **fnbattleship**, which plays on the real server. **`make build/fnbounce-host`** builds **fnbounce** the same way, with TCP: it is compiled as for CMOC, with stand-ins for CMOC's `cmoc.h` and `coco.h` in `fujinet/bounce/host`, so that it takes the same paths as on the board, and plays against a real server.

Reference Pages 3

The reference pages follow, in four sections. Within a section they are in alphabetical order. Each page has the same parts, where they apply: **Name**, **Syntax**, **Description**, **Options**, **Restrictions**, **Examples**, **Files**, and **See Also**. The commands that came with NitrOS-9 are not described here; use **man** on the board for those.

Section 1: Commands

This section describes the commands added to NitrOS-9 Level 2 for the TFR/911H. The commands that come with NitrOS-9 are described by **man** on the board, and in the NitrOS-9 documentation.

Name	Description
beep(1)	Ring the bell
clear(1)	Clear the screen
click(1)	Turn keyclicks on or off
fnbattleship(1)	Fuji Battleship, through the FujiNet
fnbounce(1)	Bounce World, through the FujiNet
fnconfig(1)	FujiNet CONFIG: the FujiNet's status, hosts, device slots, and WiFi
fnnews(1)	The news, through the FujiNet
fnstocks(1)	Stock quotes, through the FujiNet
fnweather(1)	The weather, now and for the week, through the FujiNet; fnwx
fujinet(1)	Commands for the FujiNet: fnstatus, fnlisthosts, fnlistdevs, fnsetdevfile, …
grep(1)	Print lines that contain a string
info(1)	Show what the system and the hardware know about themselves
pwd(1)	Print the data or execution directory
tsh(1)	The shell: line editing, history, completion, and a search path
unix(1)	Unix-style names for OS-9 commands: ls, cd, cat, cp, mv, rm, ps, df, …
vi(1)	A small vi, also called vim

Name **beep** – ring the bell

Syntax **beep**

Description The **beep** command writes a bell character (`$07`) to standard output. The terminal the tether runs in rings its bell, and the TFR/911H firmware, which sees the character go out through the 6850, plays an 880 Hz tone for 150 ms through the board's speaker, if one is connected and bit 1 of the control register at `$FF42` is set (it is, at power on).

Any program that prints `^G` makes the same sound.

Examples To sound the bell when a long command finishes:

```
OS9:copy /dd/bigfile /dd/backup; beep
```

See Also **click**(1), **sn76489**(4), **tfr911h**(4)

Name **clear** – clear the screen

Syntax **clear**

Description The **clear** command sends the ANSI sequences that move the cursor home and clear the screen of the terminal the tether runs in.

See Also **unix**(1)

Name **click** – turn keyclicks on or off

Syntax **click** `on|off`

Description When keyclicks are on, the TFR/911H's speaker gives a short tick each time a key reaches OS-9. The **click** command turns them on or off by setting or clearing bit 0 of the control register at `$FF42`. They are on at power on. With no argument, or any other, **click** prints its usage.

The `TCL>` console has a **click** command of its own, which does the same before an OS is booted; see **tcl**(8).

Restrictions **tetris**(6) turns keyclicks off while it runs, and puts them back as they were when it ends.

See Also **beep**(1), **sn76489**(4), **tfr911h**(4)

Name

fnbattleship – Fuji Battleship, through the FujiNet

Syntax

fnbattleship

Description

The **fnbattleship** command plays Fuji Battleship, a game of sea battle for up to four players, on a server on the Internet, against people on other computers or the server's robots. Each player places five ships on a board of ten squares by ten; then they take turns, and a shot at a square is a shot at that square on every other player's board. The last with a ship afloat wins.

fnbattleship starts with its title, which shows its version and the day it was built. The first time, it asks for a name, which the FujiNet keeps for the games of its lobby, and shows how to play. Then it lists the server's tables: some for people, and some with robots (`ai - 1 on 1`, `ai - 2 bots`, `ai - 3 bots`). At a table the players get ready, and when all are, a count-down starts; then each places the ships, and the turns begin. Your board, with your ships, is on the left, the others on the right, or each in a quarter of the screen when there are three or four. Hits burn, misses splash, and under or beside each board is that player's fleet, crossed out as it sinks. Between the boards is whose turn it is. The server gives each player a time to move; the last twenty seconds count down at the bottom right, and then the server moves for you.

Up, Down	In the list of tables, choose one; Space joins it. R lists them again, H shows how to play, N changes your name, S turns the sound on or off, and Q quits.
Space	At a table, get ready, or not.
R	While placing a ship, turn it; the arrows move it, and Space places it.
A–J, 1–0	On your turn, aim at that column or row (0 is row 10); the arrows aim too, and Space fires.
Esc	The menu: Q leaves the table, H shows how to play, S turns the sound on or off, and Space goes back.

At the end of a game, Space goes on to the next.

Restrictions

NitrOS-9 waits while the FujiNet fetches each reply from the server, which takes a second or more. The screen must be at least 80 columns by 24 lines, and show UTF-8. The sounds are played on the SN76489 (**sn76489**(4)).

Files

`/FujiNet/00010100.key`	On the FujiNet's SD card: your name, for the games of the FujiNet's lobby.
`/FujiNet/00010105.key`	The server and the table you are at, so that **fnbattleship** goes back to it after a reset; leaving the table forgets it.
`/FujiNet/e41c0500.key`	Whether the sound is on, and that how to play has been shown.

See Also

fnbounce(1), **fujinet**(4), **sn76489**(4)

Name

fnbounce – Bounce World, through the FujiNet

Syntax

fnbounce

Description

The **fnbounce** command is a client of Bounce World, a world of shapes that bounce off one another and off its edges. A server keeps the world, which is made of the screens of all its clients side by side, so that a shape that leaves one computer's screen comes onto the next one's. **fnbounce** starts with its title screen, which shows its version and the day it was built, and where the server, a name, and the width of the world are set; the FujiNet keeps them in app keys from one run to the next. Then it shows the kinds of shape, and then its part of the world, ten times a second.

S	Type the server: *host*`:`*port*, where the port is the server's framed TCP port, 9003 unless it was set otherwise; `tcp://` is added if it is left out.
N	Type a name, of up to seven characters.
W	Change the world's width between 40 and 80. At 40 each place of the world is two columns of the terminal, so that the shapes are round; at 80 a place is a column, and the world is twice as wide, with narrower shapes.
Esc	Quit.

Any other key goes on. While a setting is typed, Esc, or typing nothing, leaves it as it was. In the world:

I	Show or hide two lines of information: the name, how many clients there are and how many shapes of each size, the world's size (in reverse video while it is frozen), and the keys.
W	Show or hide the names of the clients.
1–5	Add a shape of that size.
+, -	Speed the world up; slow it down.
F	Freeze the world, or let it go again.
R	Reset the world.
C	Show the screen in reverse video, or not.
L	Flash the screen when shapes collide, or not.
S	Make a sound when shapes collide, or not.
Q, Esc	Quit. Ctrl/C is the same.

The world is shared: 1–5, +, -, F, and R change it for every client.

Restrictions

The server must be of version 2.3 or later; its other port, 9002, is for older clients, whose replies have no length, and **fnbounce** cannot read them. The screen must be at least 80 columns by 24 lines, and show UTF-8. The sound is a burst on the noise voice of the SN76489 (**sn76489**(4)).

Examples

To use the server at `bouncy.diller.org`, type S at the title screen, then

```
bouncy.diller.org:9003
```

See Also

fnnews(1), **fnstocks**(1), **fujinet**(4)

Name **fnconfig** – FujiNet CONFIG for NitrOS-9

Syntax **fnconfig**

Description The **fnconfig** command is a full-screen program for managing the FujiNet, as the CONFIG program does on other computers. It has four pages, chosen with the number keys:

1 Status	The FujiNet's firmware version, host name, WiFi network, and IP settings.
2 Hosts	The eight host slots. Up and Down pick one; E edits it, M mounts it, and U unmounts it.
3 Devices	The four DriveWire device slots, /X0–/X3. B browses the slot's host for an image, E edits the path, M mounts the image (it asks whether read-only or read/write), and U ejects it.
4 WiFi	S scans for networks; E enters a network's name. Either then asks for the password.

On every page R reads everything from the FujiNet again, I shows the program's and the firmware's versions, ? shows help, and Q quits. The FujiNet saves each change when it succeeds.

Restrictions When NitrOS-9 was booted from the FujiNet (**l2fn**), /X0 is /DD: **fnconfig** will not change device slot 0 or its host, and asks before changing the WiFi, which may drop the connection the disk depends on.

The screen must be at least 80 columns by 24 lines. **fnconfig** was written for a Color Computer 3's window; on the TFR/911H it is built to send ANSI escape sequences instead.

See Also **fujinet**(1), **fujinet**(4)

Name **fnnews** – the news, through the FujiNet

Syntax **fnnews**

Description The **fnnews** command reads the news from the FujiNet news server, which gathers stories from many sources and formats them to fit the screen. It is the Color Computer's news reader, from `fujinet-news` (see Chapter 2). It begins with nine topics: top stories, world news, business, science, technology, health, entertainment, politics, and sports. Up and Down choose one, Enter shows its headlines, six to a page, newest first, each with its time, and I shows where the news comes from: the server, and the program's versions:

Up, Down	Choose a headline; past the first or the last, turn the page.
Left, Right	The previous or the next page of headlines.
G	Go to a page, by its number.
Enter	Read the story.
Esc	Back to the topics.

A story is shown a page at a time, its headline at the top and the page number at the bottom:

Up, Down	The previous or the next page.
I	The story's date and source.
Esc	Back to the headlines.

Esc at the topics quits. Ctrl/C is the same as Esc.

Restrictions Each page is fetched when it is shown, and NitrOS-9 stops while the FujiNet fetches it, for a second or two. If a fetch fails, the page is empty; going back and choosing it again fetches it again. The screen must be at least 80 columns by 24 lines.

See Also **fnstocks**(1), **fnweather**(1), **fujinet**(4)

Name **fnstocks** – stock quotes, through the FujiNet

Syntax **fnstocks**

Description The **fnstocks** command follows up to ten stocks. Their symbols are in two columns in the middle of the screen, in alphabetical order, and across the top a ticker scrolls each one's price and the day's change. The quotes come from Finnhub (`finnhub.io`), and are fetched again every five minutes. The FujiNet keeps the list, in its app keys, from one run to the next. **fnstocks** is `fujinet-stocks`, which runs on many computers (see Chapter 2).

Arrows, 1–9, 0	Select a stock; 0 is the tenth.
E	Type a symbol into the selected place.
L	Look up a symbol by a name, or part of one, and put the one chosen in the selected place.
D	Delete the selected stock.
I	The selected company: its name, exchange, country, industry, market capitalization, shares, telephone number, and web site.
R	Fetch the quotes now.
H, S	Hide the list, leaving the ticker; show it again.
Esc	Quit. Ctrl/C is the same.

Restrictions Each quote takes about two seconds to fetch, and NitrOS-9 stops while the FujiNet fetches it. Finnhub's free quotes may be some minutes old. The screen must be at least 80 columns by 24 lines.

See Also **fnnews**(1), **fnweather**(1), **fujinet**(4)

Name

fnweather, **fnwx** – the weather where you are, through the FujiNet

Syntax

fnweather
fnwx

Description

The **fnweather** command shows the weather where you are, now and for the week. It finds where you are from your network's public address (from `ip-api.com`), or from the name of a city, and fetches the weather there from Open-Meteo.

Its screen shows the sky as a picture—sun or moon, clouds, rain, a storm, snow, or fog—beside the temperature, in large figures colored from blue for cold to red for hot, the description, what it feels like, and today's high and low. To the right are the humidity and the clouds, as bars, the dew point, the pressure, the visibility, the wind, today's ultraviolet index, colored by the WHO's scale, and the times of sunrise and sunset. Below are the next eight days, each with its picture, its high and low, and its chance and amount of rain. F shows the forecast in detail, four days to a page.

fnweather keeps what it fetched, with the place and the units, in `/DD/SYS/fnweather.dat`. It shows that at once, and fetches the weather again when it is half an hour old, at the start or while it is showing it, which it says on the line above the keys. It uses Fahrenheit, miles, and inches in the United States, Liberia, and Myanmar, and Celsius and kilometres elsewhere, until U changes them.

R	Fetch the weather now.
F	The forecast: N and B, or the arrows, turn its two pages, and W or Esc comes back.
U	Change between metric and imperial units.
L	Choose the place: type a city, or press Enter for where you are.
I	Show where the data comes from, and the credits.
Q, Esc	Quit. Ctrl/C is the same.

fnwx is the first **fnweather**, in assembly, kept as the example of Chapter 2: the weather now, in words and numbers, with R, U, L, D (the data received), I, and Q.

Restrictions

Fetching the weather takes some seconds, and NitrOS-9 stops while the FujiNet answers each request. The screen must be at least 80 columns by 24 lines, and show UTF-8.

Files

`/DD/SYS/fnweather.dat`	The cache: the place, the units, and the last weather fetched.

See Also

fnconfig(1), **fnnews**(1), **fnstocks**(1), **fujinet**(4)

Name

fnstatus, **fnlisthosts**, **fngethost**, **fnsethost**, **fnmount**, **fnlistdevs**, **fngetdevfile**, **fnsetdevfile**, **fnmountimg** – manage the FujiNet

Syntax

fnstatus
fnlisthosts
fngethost *host_slot*
fnsethost *host_slot host*
fnmount *host_slot*
fnlistdevs
fngetdevfile *device_slot*
fnsetdevfile *device_slot host_slot mode path*
fnmountimg *device_slot* [*mode*]

Description

These commands talk to the FujiNet through `/N1`. A FujiNet has eight *host slots*, each a place images come from: a TNFS server, or `SD` for its own SD card. It has four DriveWire *device slots*, each an image from a host, which NitrOS-9 sees as the disks `/X0`–`/X3`. Slots are numbered from 0, as the FujiNet numbers them: device slot 0 is `/X0`. A *mode* is 1 for read-only or 2 for read/write.

fnstatus Print the FujiNet's firmware version, host name, WiFi network, IP address, gateway, netmask, and name server, and whether the WiFi is connected.

fnlisthosts List the host slots.

fngethost Print one host slot.

fnsethost Set one host slot, keeping the others.

fnmount Mount a host, so that its images can be used.

fnlistdevs List the device slots: each one's host slot, mode, and image.

fngetdevfile Print one device slot's image.

fnsetdevfile Put the image *path*, from host *host_slot*, in a device slot. The FujiNet saves it.

fnmountimg Mount a device slot's image (read-only, unless *mode* is 2).

Examples

Use an image on the FujiNet's SD card (host slot 1) as `/X1`:

```
OS9: fnsetdevfile 1 1 2 /games/disk.dsk
OS9: fnmountimg 1 2
OS9: dir /x1
```

See Also

fnconfig(1), **fujinet**(4)

Name

grep – print lines that contain a string

Syntax

grep [`-i`] [`-n`] *pattern* [*path*]

Description

The **grep** command prints each line of *path*, or of standard input if no path is given, that contains *pattern*. The pattern is a plain string, not a regular expression. It may be given in double quotes, which it needs if it contains spaces, or without them.

This is NitrOS-9's **grep**, changed in two ways: the pattern need not be quoted, and it ends cleanly at the end of a file, where the original reported a spurious error.

Options

`-i` Ignore case.

`-n` Put each line's number in front of it.

Examples

```
OS9:grep -n total sample.c
OS9:dir -e ! grep -i tetris
```

See Also

unix(1)

Name

info – show what the system and the hardware know about themselves

Syntax

info [`-p`]

Description

The **info** command prints a report in eight parts:

System	The OS name and version, the machine, the kernel's level and version and the CPU it was built for, the startup program, disk and console, the sizes of the device and interrupt tables, and the date. These come from the `Init` module.
Memory	The RAM, what is free (and the largest piece that is free), and what is used by modules and by everything else, from the system's memory block map.
Processes	Every process: its number, its parent, its user, its priority, its memory, its state, and the program it runs. The one marked `*` is **info** itself.
Devices	Every device descriptor in memory, with its file manager, driver, port, and drive, and how many paths are open on it. A device that nothing is using is not attached, and shows `-`.
Disks	Each disk drive (once, though several descriptors may name it): its name, the date it was made, its size, and its free space, from its sector 0 and its allocation bitmap.
Interrupts	The entries in the IRQ polling table, with the driver and device each belongs to.
Modules	The names of the modules in memory, and their total size.
Hardware	The TFR/911H firmware's own report: the board, the firmware's build date, the RP2350B's clock, memory and ID, the 6309's measured speed and how busy it is, the interrupt rate, the disk, console, sound, and FujiNet counters, whether the clock was set, and the uptime.

The kernel's tables are in the system's own address space. **info** reads them as NitrOS-9's **procs**, **devs**, and **mdir** do: it gets the system process's descriptor with `F$GPrDsc`, and uses the memory map in it to copy what it needs with `F$CpyMem`. The hardware report comes from the firmware's info port; see **tfr911h**(4).

Options

`-p`	Plain text: leave out the escape sequences that make the headings bold, for saving the report to a file.

Restrictions

The RP2350B's temperature sensor reads nonsense on some boards, so the chip's temperature is shown only when its reading is believable.

On firmware without the info port, the hardware part says so and is otherwise empty.

Examples

Part of a report, taken with **tetris**(6) running in the background:

```
OS9:info
NitrOS-9/6809 Level 2 V03.3.0 on the TFR/911H

System
  OS          NitrOS-9/6809 Level 2 V03.3.0
  Machine     Tandy Color Computer 3, as the TFR/911H pretends to be
  Kernel      Level 2, version 3.3.0, built for the 6809
  Startup     runs SysGo, disk /DD, console /Term
  Tables      39 device slots, 21 IRQ polling slots
  Date        2026/10/17 01:25:15

Memory
  RAM         248K in 31 blocks of 8K
  Free        160K (largest piece 152K)
  In use      24K modules, 64K system and processes

Processes
   ID  Parent  User  Prio  Memory  State      Program
   1     0       0     255   0                (the system)
   2     1       0     128   768     waiting    Shell
   3     2       0     128   4K      waiting    tetris
  *4     2       0     128   6K      running    info
```

```
Devices
  Device    Type  Manager  Driver    Port   Drive  Users
  /Term       SCF   SCF      sc6850    $FF14         4
  /DD         RBF   RBF      EmuDsk    $FF80  0      5
  /H0         RBF   RBF      EmuDsk    $FF80  0      -
  /Pipe       Pipe  PipeMan  Piper     $0000         -

Disks
  /DD = /H0
              "tfr911-level2.dsk", made 2026/10/17
              24.4M (99999 sectors), 24.0M free (98%)

Hardware (from the TFR/911H firmware)
  Board       TFR/911H: a Hitachi 6309 run by a Raspberry Pi RP2350B
  Pico        RP2350B at 250 MHz, 520K SRAM
  6309        3.24 MHz (measured), 70% busy, in 6809 mode
  Interrupts  59 IRQs/s: GIME VBORD 60 Hz clock, 6850 ACIA
  Sound       SN76489 at $FF41 (182 writes), PWM audio on GPIO 8, keyclick off
  Uptime      Pico 0m 56s, NitrOS-9 0m 54s
```

To keep a report:

```
OS9:info -p >/dd/sysinfo
```

See Also **tfr911h**(4); NitrOS-9's **mdir**, **mfree**, **procs**, **devs**, **irqs**, **free**

Name **pwd**, **pxd** – print the data or execution directory

Syntax
pwd
pxd

Description An OS-9 process has two current directories. The **pwd** command prints the *data* directory, where file names that do not begin with **/** are looked for, and which **cd** (or **chd**) changes. The **pxd** command prints the *execution* directory, where commands are looked for, which **chx** changes.

Both are NitrOS-9's **pd**, built twice.

Examples
```
OS9:cd /dd/games
OS9:pwd
/DD/GAMES
OS9:pxd
/DD/CMDS
```

See Also **unix**(1)

Name

tsh – the shell, with line editing, history, completion, and a search path

Syntax

tsh [*file*]

Description

tsh is the shell you type to after booting: the last line of `startup` starts it. It reads each line itself, so that you can move about in the line and change it, call back the lines you typed before, and have names finished with Tab. Then it hands the line to the OS-9 Shell to run. So commands are written as for the Shell: `!` for a pipe, `<`, `>`, `>>` and the rest for redirection, `#`*n*`K` for memory, `&` and `;`, parentheses, and procedure files all work as they always have.

A command that is neither in memory nor in the execution directory is looked for in each directory of the *path* in turn; see **path** below. `/DD/SYS/tshrc` sets it when **tsh** starts.

`exit` leaves **tsh**. After booting, that gives the OS-9 Shell; type **`tsh`** to come back. Given a *file*, **tsh** runs the commands in it and exits; and if its standard input is not a terminal, it runs the commands it reads there.

Editing

Left Right, `^B ^F`	Move a character.
`^A ^E`	To the start or the end of the line.
Esc `b`, Esc `f`	Back or on a word.
Backspace	Delete the character before the cursor.
`^D`	Delete the character under the cursor.
`^W`	Delete back to a space: the word before the cursor.
Esc Backspace, Esc `d`	Delete a word back, or on.
`^K ^U`	Delete to the end of the line, or to its start.
`^X`	Delete the whole line.
`^Y`	Put back what was deleted last.
`^T`	Swap the two characters before the cursor.
`^L`	Clear the screen and draw the line again.
`^C`	Give up the line.
Return	Run it, wherever the cursor is.

A line longer than the screen scrolls sideways; a < at its left or a > at its right shows that there is more.

History

Up Down, `^P ^N`	The line before, or after.
Shift/Up Shift/Down	The line before, or after, that starts as this one does, up to the cursor: type **`cop`** and press Shift/Up for the last **copy**.
`^R`	Search back: type some of the line you want, and `^R` again for an older one. Return runs it; any other key leaves it to be edited; `^C` gives up.

The last 100 lines are kept in `/DD/SYS/tsh_history`, so they are there after booting again. **tsh** adds them to the file ten at a time, and when it exits; quitting the tether loses none, since **tsh** goes on running on the board, but pressing RESET loses those not yet written.

Completion Tab finishes the word before the cursor. At the start of a command it is a command's name: built in, an alias, or a file in the execution directory or a directory of the path. Anywhere else it is a file's name, in the data directory or in the directory the word gives. A directory is followed by `/`, anything else by a space. When several names fit, Tab adds what they begin with in common, and a second Tab lists them.

Built-in commands

cd [*dir* | `-`], **chd**	Change the data directory: to `/DD` without one, back to the last with `-`.
chx *dir*	Change the execution directory.
path [*dir* …]	Show the path, or set it. *+dir* adds a directory at the end, *–dir* takes one out, **–** empties it. A relative *dir* is made full from the data directory.
`PATH=`*dir*`:`*dir*	Set the path (spaces separate them too).
`PROMPT=`*text*	Set the prompt (`OS9:` to begin with): `%d` is the data directory, `%w` the last name in it, `%h` the number the next line will have in the history, `%e` the last command's exit status, `%t` the time.
`COLS=`*n*	The screen's width (80).
set	Show `PATH`, `PROMPT`, and `COLS`.
history [*n* | `-c`]	List the history, or the last *n* lines of it; `-c` forgets it all.
alias [*name* [*text*]]	Make *name*, as the first word of a command, stand for *text*. With only a name, show it; with nothing, list them all. **unalias** *name* removes one.
source *file*, **.**	Run the commands in *file* in this **tsh**.
which *name* …	Say what each name is: an alias, built in, in memory, in the execution directory, or where on the path.
w, **wait**	Wait for the commands in the background to finish.
kill, **setpr**	As in the Shell.
x -x, **p -p**, **t -t**	As in the Shell: stop at an error, show no prompt, echo each line; or not.
ex *command*	Run *command*, then leave **tsh**.
exit [*status*]	Leave **tsh**.
`*`	A comment, to the end of the line.

How it runs a command For each command between `;` and `&`, **tsh** puts an alias's text in place of its name, does a built-in command itself, and starts `Shell` *command* for any other, waiting for it unless `&` followed. The Shell's own built-in commands are done by **tsh**, because a Shell given only **`chd /dd`** would change its own directory, not **tsh**'s, and then wait for more input. A command name found on the path is loaded by **tsh** with `F$Load`, so the Shell finds it in memory, and unlinked when the command has finished. A procedure file on the path is given to the Shell as a full pathlist.

tsh reads keys through a path of its own to the terminal, with echo, paging, and the keyboard's signals turned off for that path alone, so the Shell and other programs see the terminal as they always have. On the TFR/911H's console it sets bit 7 of `$FF42` while you edit a line, for the arrow keys (see **tfr911h**(4)), and clears it before running the line. It draws with backspaces, carriage returns, and printing characters, so any terminal will do; `^L` alone sends ANSI sequences.

Ctrl/C and Ctrl/E reach a running command as they always have. If the command has not used the terminal since **tsh** did, they come to **tsh** instead, and it aborts the command with `S$Abort`, through its Shell.

Restrictions

The tether passes on the arrow keys, and Shift with the up and down arrows, but not Home, End, or Delete: use `^A`, `^E`, and `^D`. Over a network, from a terminal that sends them as ANSI sequences, those keys work too.

There are no variables but `PATH`, `PROMPT`, and `COLS`, and no wildcards: the Shell has none either. A line is at most 250 characters. Each command costs a process for its Shell, about 8K of memory while it runs. The Shell's `i=` is not taken.

Examples

Find out where commands come from, and add a directory to the path:

```
OS9:which dir ll fnweather
```

```
dir: in the execution directory
ll: in the execution directory
fnweather: in the execution directory
OS9:path +/H0/BIN
OS9:path
/DD/CMDS /H0/CMDS /H0/BIN
```

An alias, and a prompt that shows the directory:

```
OS9:alias lsl dir -e
OS9:lsl ! more
OS9:PROMPT="%d: "
/DD: cd cmds
/DD/CMDS:
```

Files

`/DD/SYS/tshrc`	Commands run when **tsh** starts: the path, the prompt, aliases.
`/DD/SYS/tsh_history`	The history.
`/DD/startup`	Ends with **`tsh </term`**, to start it after booting.
`nitros9-l2/tsh/tsh.c`	The source. **`make build/tsh-host`** builds it for the computer the tether runs on, where commands go to `/bin/sh`, for testing.

See Also

unix(1), **tfr911h**(4), **nitros9-l2**(8)

Name

unix – Unix-style names for OS-9 commands

Syntax

ls [`-l`] [*path*] **cd** *path* **cat** *path* …

Description

The TFR/911H's NitrOS-9 knows the names a Unix user reaches for first. Most are *aliases*: tiny modules that run the OS-9 command that does the job, passing along their arguments. Two are built into the Shell and **tsh**, and the rest are real commands. **`man unix`** on the board lists them.

Name	Runs	What it does
ls [`-l`]	**dir** [`-e`]	List a directory; `-l` gives the long form
ll	**dir -e**	List a directory, long form
cat	**list**	Print a text file
cp	**copy**	Copy a file
mv	**rename**	Rename a file or directory
rm	**del**	Delete files
mkdir	**makdir**	Make a directory
rmdir	**deldir**	Delete a directory and what is in it
ps	**procs**	List processes; `-e` lists everyone's
df	**free**	Show free space on a disk
less	**more**	Show a file a screen at a time
man	**help**	Show help for a command
uname	**echo**	Say what this is
vim	**vi**	The editor; see **vi**(1)
cd	built in	Change the data directory (the same as **chd**)
exit	built in	Leave this shell
pwd, **pxd**	**pd**	See **pwd**(1)
grep, **clear**		See **grep**(1), **clear**(1)

An alias puts any extra arguments of its own in front of yours, and then chains to the real command with `F$Chain`, so it costs no extra process. **cd** and **exit** are added to the table of built-in commands in a copy of the Shell's source.

Restrictions

These are OS-9 commands under Unix names, and they behave as OS-9 commands. **mv** cannot move a file to another directory. **rmdir** deletes the directory's contents too, after asking. **rm** takes no options.

The Shell's syntax is OS-9's, not the Bourne shell's: the pipe symbol is `!`, >> redirects standard error (there is no append), <> and >>> redirect several paths at once, and `&` runs a command in the background.

Examples

```
OS9:ls -l /dd
OS9:mkdir /dd/src
OS9:cp notes /dd/src/notes
OS9:cd /dd/src
OS9:ps
```

Files

`nitros9-l2/unix.tcl` The list of aliases. Add a line to it and rebuild to make another; see **nitros9-l2**(8).

See Also

grep(1), **pwd**(1), **tsh**(1), **vi**(1)

Name

vi, **vim** – a small vi

Syntax

vi [`-w` *cols*] [`-h` *rows*] [*file*]

Description

The **vi** command is a screen editor in the manner of the Unix **vi**, with most of the commands people use without thinking about them. It starts in *normal* mode, where keys are commands. The insert commands enter *insert* mode, where keys are text, until you press Esc. A colon starts a *command line* at the bottom of the screen.

The bottom line shows messages, `-- INSERT --` in insert mode, and the cursor's line and column. Lines longer than the screen are not wrapped: the view scrolls sideways.

Most normal-mode commands take a count in front: **`3dd`** deletes three lines, **`5j`** moves down five, **`2d3w`** deletes six words.

Moving

`h j k l`	Left, down, up, right. The arrow keys too.
`w b e`	To the next word, back a word, to the end of the word. `W B E` do the same with words that are anything but blanks.
`0 ^ $`	To the start of the line, its first non-blank, its end.
`G`, `gg`	To the last line, or to line *n* with a count; to the first line.
`H M L`	To the top, middle, or bottom of the screen.
`f`*c* `F`*c* `t`*c* `T`*c*	To the next (`f`) or previous (`F`) *c* on the line, or just before it (`t`, `T`). `;` repeats; `,` repeats the other way.
`%`	To the bracket that matches the next `( [ { } ] )`.
`{ }`	Back or on a paragraph (to a blank line).
`+ - Return`	Down or up a line, to its first non-blank.
`^F ^B`	Forward or back a screen. Page Down and Page Up too.
`^D ^U`	Down or up half a screen.

Searching

`/`*text*, `?`*text*	Find *text* forward or backward, wrapping around the file.
`n N`	Find it again, the same way or the other.
`*`	Find the word under the cursor.

Inserting

`i a`	Insert before, or append after, the cursor.
`I A`	Insert at the line's first non-blank, or append at its end.
`o O`	Open a new line below, or above.

In insert mode, Backspace deletes the character before the cursor (and can join lines), `^W` the word before it, `^U` the line before it; Return splits the line, Tab inserts a tab, and the arrow keys move the cursor. Esc goes back to normal mode.

Changing

`x X`	Delete the character under, or before, the cursor.
`D C`	Delete, or change, to the end of the line.
`s S`	Change a character, or the whole line.
`r`*c*	Replace the character with *c* (`r` Return splits the line).
`J`	Join this line and the next.
`~`	Switch the case of the character, and move on.
`>> <<`	Indent the line by a tab, or take one away.
`p P`	Put the last deleted or yanked text after, or before, the cursor (below or above, for whole lines).
`u`	Undo the last change. Again and again, back to the start.
`.`	Do the last change again.

Operators d (delete), c (change), y (yank), > and < (indent) work on the text that any motion moves over: **dw**, **c$**, **y3j**, **dG**, **d/end**, **>}**. Doubled, they work on whole lines: **dd**, **cc**, **yy**, **>>**. Y is yy.

Other

ZZ	Write the file if it was changed, and quit.
ZQ	Quit without writing.
^G	Show the file's name, length, and where you are in it.
^L	Redraw the screen.

Command line

:w [*file*]	Write the file, or write to *file*.
:q, :q!	Quit; ! quits even if there are unsaved changes.
:wq, :x	Write and quit (:x writes only if there were changes).
:e[!] [*file*]	Edit another file, or the same file again.
:r *file*	Read *file* in below the current line.
:*n*, :$	Go to line *n*, or the last line.
:[*range*]s/*old*/*new*/[g]	Replace *old* with *new*, once in each line or everywhere (g).
:[*range*]d	Delete lines.
:help	A reminder of these.

A *range* is a line number, . (this line), $ (the last), two of them with a comma (2,10), or % (every line). Without one, a command works on the current line.

Options

-w *cols*	The terminal's width (default 80).
-h *rows*	The terminal's height (default 24).

Restrictions

Searches are for plain text, not regular expressions. There is one register, there are no marks, no visual mode, no :set, and no redo. The text must fit in memory: about 25K. Undo remembers as many changes as fit in its own space, and forgets the oldest first.

OS-9 text files end their lines with a carriage return. **vi** reads files that end them with CR, LF, or CR LF, and writes CR.

The arrow keys work in insert mode because **vi** sets bit 7 of $FF42 while it runs, so that the firmware sends them untranslated (see **tfr911h**(4)). If **vi** is killed, they stay that way until the next **vi** ends.

Examples

Change every foo to bar, write, and quit:

```
:%s/foo/bar/g
:wq
```

Delete from here to the line that contains end, then put it back:

```
d/end
u
```

See Also

unix(1), **tfr911h**(4)

Section 4: Special Files

This section describes the hardware the TFR/911H firmware presents to NitrOS-9: the addresses in the 6309's I/O page that a program can read and write, and what they do.

Name	Description
fujinet(4)	The FujiNet: its wiring, the becker port, its disks, and booting from it
sn76489(4)	The sound chip
tfr911h(4)	The hardware the firmware emulates: memory, interrupts, console, disk, clock, keyboard

Name

fujinet – the FujiNet, its disks, and booting from it

Description

A FujiNet is a network adapter for old computers: an ESP32 with WiFi and an SD card. On the TFR/911H it runs the FujiNet firmware for the Color Computer, built for the board `fujinet-coco-devkitc-centipede`, which speaks DriveWire over a 3.3-volt serial link at 921,600 baud. It is wired as the Centipede's FujiNet hat is:

RP2350B	ESP32	Signal
GPIO 12	GPIO 33	Data to the FujiNet (UART0 TX to UART1 RX)
GPIO 13	GPIO 21	Data from the FujiNet (UART1 TX to UART0 RX)
GND	GND	

The becker port The firmware passes the link to the 6309 as a becker port, the kind CoCo emulators give DriveWire:

`$FF45`	Read: bit 1 is set when a byte from the FujiNet is waiting.
`$FF46`	Read: the next byte from the FujiNet. Write: a byte to the FujiNet.

Emulators have the becker port at `$FF41`, where the TFR/911H has its sound chip, so NitrOS-9's DriveWire modules are built with `BECKBASE` set to `$FF45`. The firmware keeps 4K of what the FujiNet sends, so nothing is lost while the 6309 is busy.

Devices `/X0`–`/X3` are the FujiNet's four device slots, as DriveWire disks (`rbdw`). `/N` and `/N1`–`/N5` are DriveWire's network devices (`scdwv`); the **fujinet**(1) commands use `/N1`, and its `SS.Fuji` SetStat, which sends a request to the FujiNet and reads its reply with interrupts masked. There is no virtual-serial poller, which would take the FujiNet's replies for itself.

Booting from the FujiNet At the `TCL>` prompt, **l2fn** mounts the image in the FujiNet's first device slot (slot 0), and boots NitrOS-9 from it: the kernel is in the firmware, as for **l2**, and its `Boot` module loads `OS9Boot` from DriveWire drive 0. That `OS9Boot` makes `/X0` the system disk, `/DD`; the tether's disk is `/H0`. To make such a disk, put it on the FujiNet, and boot it:

```
$ cd v4/nitros9-l2
$ make fujinet-disk
$ curl -X MKCOL http://fujinet/dav/tfr911
$ curl -T level2-fn.dsk http://fujinet/dav/tfr911/level2-fn.dsk
```

Then, with NitrOS-9 booted with **l2**, put the image in device slot 0 (host slot 1 being the SD card), and boot it:

```
OS9: fnsetdevfile 0 1 2 /tfr911/level2-fn.dsk

$ BOOT=l2fn sh run-l2.sh
```

make fujinet-disk copies `level2.dsk`, with your files, to `level2-fn.dsk`, and gives the copy the FujiNet's `OS9Boot`. The FujiNet's web server offers its SD card by WebDAV at `/dav`. Chapter 2 describes the FujiNet, the link, and programming it at length.

Restrictions

At power on, the FujiNet puts its own CONFIG disk in device slot 0; that is why **l2fn** mounts the slot first.

The FujiNet keeps the image file open. After replacing the file, mount it again (**l2fn** does, as does `fujinet mount 1` at `TCL>`).

The FujiNet's firmware must read the version byte that NitrOS-9 sends after `OP_DWINIT`; firmware that does not loses the command after it, and the 6309 waits for ever. See `v4/nitros9-l2/fujinet/README.md`.

Files

`v4/tfr911h/v4_fujinet.h`	The becker port and the link.
`v4/nitros9-l2/fujinet/`	The DriveWire modules, the **fn** commands and their libraries, and how to set up the FujiNet.

See Also

fnconfig(1), **fujinet**(1), **tfr911h**(4), **tcl**(8)

Name

sn76489 – the sound chip

Syntax

`$FF41`, write only

Description

The TFR/911H firmware emulates a Texas Instruments SN76489 sound generator at `$FF41`, the address of the one in the Color Computer's Game Master Cartridge. It has three square-wave tone voices and a noise voice, each with sixteen volumes. The firmware mixes them at 31,250 samples a second, with the keyclick and bell sounds, and plays the result as pulse-width-modulated audio on GPIO 8 (see Appendix A).

The chip is programmed one byte at a time. A byte with bit 7 set *latches* a voice and a register and gives the low four bits of the value; a byte with bit 7 clear gives the high six bits of a tone's period:

Byte	Meaning
`1 vv 0 dddd`	Voice `vv` (0–2): low 4 bits of its period
`0 0 dddddd`	The high 6 bits of the latched voice's period
`1 vv 1 aaaa`	Voice `vv` (0–3): attenuation, 2 dB a step; `1111` is off
`1 11 0 0 f rr`	The noise voice: `f`=1 white noise, 0 periodic; `rr` its rate, 0–2, or 3 to follow voice 2

A tone's period N (1–1023) gives a frequency of $3{,}579{,}545 / (32 \times N)$ Hz, so A440 is $N = 254$, and the lowest note is about 109 Hz.

The control register at `$FF42` is read and write:

bit 0 Keyclicks on (see **click**(1)).

bit 1 The bell sounds on `^G` (see **beep**(1)).

bit 7 Arrow keys untranslated (see **tfr911h**(4)).

Examples

In C, compiled with CMOC, from **tetris**(6):

```
#define SND (*(volatile unsigned char *)0xFF41)

void note(unsigned char voice, unsigned n) {   /* n = 3579545/32/Hz */
    SND = 0x80 | (voice << 5) | (n & 15);
    SND = (n >> 4) & 0x3F;
}

void volume(unsigned char voice, unsigned char att) { /* 15 is off */
    SND = 0x90 | (voice << 5) | att;
}
```

See Also

beep(1), **click**(1), **tfr911h**(4), Appendix A

Name

tfr911h – the hardware the TFR/911H firmware emulates for NitrOS-9 Level 2

Description

To NitrOS-9, the TFR/911H is a small Color Computer 3. The firmware's second core answers every bus cycle of the 6309: reads and writes of RAM go to the RP2350B's memory through the emulated MMU, and those of the I/O page, `$FF00–$FFFF`, go to the devices below. Locations not listed read back what was last written to them. The 6309 runs in 6809 mode.

Memory 256K of RAM, in 32 blocks of 8K. The GIME's MMU maps eight blocks into each of two tasks' 64K address spaces: `$FFA0–$FFA7` for task 0 and `$FFA8–$FFAF` for task 1. `$FF90` bit 6 enables the MMU and bit 5 the GIME's interrupts; `$FF91` bit 0 picks the task. `$FE00–$FEFF` is the constant page, the same in every map.

Interrupts `$FF92` bit 3 enables the vertical border (VBORD) interrupt, which a timer in the firmware raises 60 times a second; reading `$FF92` acknowledges it. The 6850's interrupt joins it on the 6309's IRQ line. There is no FIRQ.

Console An MC6850 ACIA at `$FF14` (status and control) and `$FF15` (data). Its transmitter is always ready; characters go to the tether at once.

Disk EmuDsk, a virtual hard disk whose sectors the tether reads and writes in a file:

`$FF80–$FF82`	The logical sector number, 24 bits.
`$FF83`	Write a command: 0 reads a sector, 1 writes one. Read the status: 0 if it worked.
`$FF84–$FF85`	The address of the 256-byte buffer.
`$FF86`	The drive number.

The bus waits while the tether moves the sector over USB, so the status is ready in the instruction after the command.

Clock `$FFC0–$FFC7` hold the century, year, month, day, day of the week, hour, minute, and second, in binary, in the format of Jeff Vavasour's CoCo 3 emulator. The tether sends your computer's local time when it connects, and the firmware keeps it going.

Sound `$FF41` and `$FF42`; see **sn76489**(4).

FujiNet A becker port at `$FF45` (status) and `$FF46` (data), for DriveWire to a FujiNet; see **fujinet**(4).

Info port The firmware's report on itself, for **info**(1). Write `$A5` to `$FF43`; when `$FF43` reads 1, each read of `$FF44` gives the report's next character, and 0 after the last. The report is lines of the form *label* `TAB` *value* `LF`.

Keyboard Keys typed in the tether reach OS-9 through the 6850. The tether turns Delete into Backspace and Enter into a carriage return. The arrow keys arrive as a Color Computer 3 sends them, unless bit 7 of `$FF42` is set:

Key	CoCo code	Bit 7 set	Note
Up	`$0C`	128	
Down	`$0A`	129	
Left	`$08`	130	The same as Backspace, as CoCo code
Right	`$09`	131	The same as Tab, as CoCo code
Page Up	—	132	Dropped, as CoCo code
Page Down	—	133	Dropped, as CoCo code

Files

`v4/tfr911h/v4_nitros9_l2.h`	The emulation.
`v4/tfr911h/v4_audio.h`	The sound chip and the mixer.
`v4/tfr911h/v4_fujinet.h`	The becker port.

See Also

info(1), **fujinet**(4), **sn76489**(4), **tether**(8)

Section 6: Games

This section describes the games. Both draw with ANSI escape sequences on the terminal the tether runs in.

Name	Description
tetris(6)	Tetris, with music
zm(6)	A Z-machine, for Zork and the other Infocom games

Name **tetris** – Tetris, with music

Syntax **tetris**

Description Pieces fall into a well ten cells wide and twenty deep. Move and turn them so they make full rows, which disappear. The game draws in color on the terminal the tether runs in, shows the next piece and a ghost of where the falling one will land, and deals the seven pieces from a shuffled bag, so no piece is ever long in coming.

Left Right, `a` `d`	Move the piece.
Up, `w`	Turn it.
Down, `s`	Drop it a row.
Space	Drop it all the way.
`p`	Pause.
`m`	Turn the music off or on.
`q`	Quit.

Clearing one, two, three, or four rows at once scores 40, 100, 300, or 1,200 points times the level plus one. Dropping a piece scores a point a row, or two with Space. Every ten rows is a new level, and the pieces fall faster, up to level 19.

With a speaker connected, the game plays *Korobeiniki* on the sound chip, faster as the levels go up, with sounds for moving, turning, landing, clearing rows, a new level, and the end of the game.

Restrictions It needs an ANSI color terminal of at least 80 by 24. It turns keyclicks off while it runs.

See Also **click**(1), **sn76489**(4)

Name

zm – play Infocom's games, and others written for the Z-machine

Syntax

zm [`-w` *cols*] [`-h` *rows*] [`-m` *kbytes*] [`-d`] *story*

Description

Infocom wrote its games for an imaginary computer, the Z-machine, and sold them with an interpreter for each real one. **zm** is a Z-machine interpreter for OS-9. It plays *version 3* story files: most of Infocom's games from 1980 to 1987 (Zork I–III, The Hitchhiker's Guide to the Galaxy, Planetfall, Enchanter, Deadline, and many more), and games written with Inform and compiled with `-v3`.

zm looks for *story*, then *story*`.z3`, and, if the name has no directory in it, the same in `/DD/GAMES`. So **`zm zork1`** finds `/DD/GAMES/zork1.z3`.

The top line of the screen is the status line: where you are, and your score and moves, or the time in the games that keep it. The text scrolls beneath it, word-wrapped to the screen's width, and pauses with `[MORE]` when a screen goes by without your typing. Games that split the screen, as Seastalker does, can.

The game's commands `save` and `restore` ask for a file name, offering the story's name with `.sav`; save files go in the data directory. `restart`, `verify`, and `quit` work as in the games' manuals.

The part of the story the game can change, its *dynamic memory*, is kept in RAM. So is as much of the rest as fits—about 40K, got by growing the process's data area with `F$Mem`. If the story is bigger than that, the rest is read from the file in 512-byte pages as the game needs them, and the least recently used page gives way. Panic, which is 37K, is held in memory whole; Zork I, at 87K, is paged, and still answers a command in under a second.

Options

`-w` *cols*	The terminal's width (default 80).
`-h` *rows*	The terminal's height (default 24).
`-m` *kbytes*	Use at most this much memory for the story (8–60).
`-d`	A dumb terminal: no status line, no escape sequences, no `[MORE]`.

Restrictions

Only version 3 stories are played; **zm** says which version a story is if it is another. There is no transcript (`script`), and there are no sounds but the two beeps of The Lurking Horror, which it plays on the sound chip.

Diagnostics

If the story does something impossible, **zm** stops with a message such as:

```
*** zm: stack underflow (PC $1A2B3) ***
```

Examples

```
OS9:zm zork1
ZORK I: The Great Underground Empire
...
West of House
You are standing in an open field west of a white house, with a boarded
front door.
There is a small mailbox here.

>open mailbox
```

To put story files on the disk, see **nitros9-l2**(8).

Files

`/DD/GAMES/*.z3`	Story files.
story`.sav`	Saved games.

See Also

nitros9-l2(8), Appendix B

Section 8: Maintenance

This section describes the programs that run on the computer the TFR/911H is plugged into, the firmware's console, and how to build the system and keep a disk up to date.

Name	Description
nitros9-l2(8)	Build NitrOS-9 Level 2 for the TFR/911H, and keep a disk up to date
tcl(8)	The firmware's console
tether(8)	The board's terminal, disk, and clock

Name

nitros9-l2 – build NitrOS-9 Level 2 for the TFR/911H, and keep a disk up to date

Syntax

make [*target*] [`DISK=`*image*] [`STORIES=`*files*] [`OS9_TOOL=`*path*]

Description

The `v4/nitros9-l2` directory builds NitrOS-9 Level 2 for the TFR/911H from a NitrOS-9 source tree, with the commands described in this manual, and makes a disk image, and another that boots from a FujiNet. It needs **lwasm** from LWTOOLS, **os9** from ToolShed, **tclsh**, **python3**, and, for the C programs, CMOC.

`make`	Build the kernel, `OS9Boot`, the commands, and `build/tfr911-level2.dsk`, and the firmware's copy of the kernel track, `../tfr911h/nitros9_l2_rom.h`.
`make install`	Bring the disk *DISK* (default `level2.dsk`) up to date: a new `OS9Boot`, and every file in `CMDS` and `SYS`. Other files are left alone.
`make stories`	Copy the story files named by *STORIES* to `/DD/GAMES` on *DISK*.
`make fujinet-disk`	Copy *DISK* to *FNDISK* (default `level2-fn.dsk`), with the `OS9Boot` that boots from the FujiNet; see **fujinet**(4).
`make build/zm-host`	Build **zm** for this computer, for testing; also `build/vi-host` and `build/tsh-host`.
`make clean`	Remove the build.

The firmware is built from `v4`: **`make build/tfr911h_v4.uf2`**, then loaded with **picotool**.

Restrictions

Quit the tether before changing the disk it is serving, and reset the board after `make install` so that it boots the new `OS9Boot`.

Examples

```
$ cd v4/nitros9-l2
$ make OS9_TOOL=~/toolshed/build/unix/os9/os9
$ make install DISK=level2.dsk
$ make stories STORIES="~/if/zork1.z3 ~/if/hhgg.z3" DISK=level2.dsk
```

Files

`rmake-l2.tcl`	Which modules go in `OS9Boot` and on the disk.
`unix.tcl`	The Unix-style aliases, and changes to the Shell and **grep**.
`modules/`	The TFR/911H's own modules: the clock, **clear**, **beep**, **click**.
`fujinet/`	The DriveWire modules and the **fn** commands.
`games/`	The C programs: **tetris**, **zm**, **vi**, **info**.
`tsh/`	**tsh**, and its `tshrc`.
`startup`	The `startup` of a new disk; it starts **tsh**. `make install` leaves a disk's own `startup` alone.
`helpmsg.unix`	The **man** pages for what was added.
`doc/manual.tex`	This manual.

See Also

tcl(8), **tether**(8)

Name **tcl** – the TFR/911H firmware's console

Syntax `TCL>` *command*

Description When the TFR/911H starts, its firmware waits at the `TCL>` prompt, a Tcl interpreter that runs on the RP2350B. Besides the Tcl language, it has these commands:

l2	Boot NitrOS-9 Level 2 from EmuDsk drive 0.
l2fn [`-n`]	Mount the image in the FujiNet's device slot 0 and boot NitrOS-9 Level 2 from it; with `-n`, boot from whatever is mounted. See **fujinet**(4).
fujinet	Ask the FujiNet the time, to show that the link works.
fujinet mount [*n*]	Mount the image in the FujiNet's device slot *n* (1–4, the FujiNet's own numbers; default 1).
fujinet stats	The link's byte and error counts.
bye	Boot TurbOS, the OS-9 Level 1 system in the firmware.
beep [*hz* [*ms*]]	Play a tone through the speaker (default 880 Hz for 200 ms), to test the wiring.
click [`on`|`off`]	Turn keyclicks on or off, and say which.

Examples

```
TCL> beep 440 1000
TCL> fujinet
FujiNet says it is 2026-10-17 18:46:20
TCL> l2
```

See Also **click**(1), **fujinet**(4), **tether**(8)

Name

tether, **run-l2.sh** – the board's terminal, disk, and clock

Syntax

sh run-l2.sh
tether `-wire` *device* [`-disks` *images*] [`-boot` *command*] [`-pass_ctrl_c`] [*other options*]

Description

The tether is a program on your computer that talks to the TFR/911H over USB. It is the board's terminal: what you type goes to the 6309, and what the 6309 prints appears in the window, UTF-8 and all (a byte of 128 or more that does not begin a UTF-8 character is shown as its Latin-1 character). It is the board's disk: it reads and writes the sectors of disk image files for EmuDsk. And it sets the board's clock when it connects.

`v4/run-l2.sh` builds the tether and runs it with the options NitrOS-9 Level 2 needs. If NitrOS-9 is already running on the board, it simply reconnects.

`WIRE`	The board's USB serial device. On macOS, the only `/dev/cu.usbmodem*` device, if there is just one.
`DISK`	The disk image for drive 0. The default, `nitros9-l2/level2.dsk`, is copied from the build the first time.
`BOOT`	What to type at `TCL>`: `l2` (the default), or `l2fn` to boot from the FujiNet.

Options

`-wire` *device*	The serial device.
`-disks` *images*	Disk image files, comma separated, for drives 0, 1, …
`-boot` *command*	A command to type at the `TCL>` prompt, such as `l2`. Ignored if the 6309 is already running.
`-pass_ctrl_c`	Send `^C` (and `^\` and `^Z`) to the 6309, and quit the tether with `^]` instead.
`-level` *n*	The NitrOS-9 level, for the tether's diagnostics.

Restrictions

Only one program can have the board's serial device open. Quit the tether before you flash the firmware or change a disk image it is serving.

Files

`v4/_log`	The tether's diagnostics, including the firmware's statistics every ten seconds.

See Also

nitros9-l2(8), **tcl**(8), **tfr911h**(4)

Connecting a Speaker A

The TFR/911H makes sound as pulse-width-modulated audio on GPIO 8 of the RP2350B: an 8-bit level carried at about 977 kHz, far above hearing. A filter and a small amplifier turn it into sound. The firmware was written for an Adafruit PAM8302, a 2.5 W class D amplifier on a small board.

A.1 Wiring

GPIO 8 is pin 1 of the 2-by-10 header J3, and is not connected to the 6309. Wire the PAM8302 like this:

J3 pin	Signal	PAM8302
2	VBUS (5 V)	Vin
6	GND	GND, and A–
1	GPIO 8	A+, through a 1 kΩ resistor, with a 10 nF capacitor from A+ to GND

The resistor and capacitor make a low-pass filter that takes out the carrier. Connect the speaker to the PAM8302's + and – terminals, and turn its volume control down before you start.

A.2 Trying It

At the `TCL>` prompt, before an OS is booted:

```
TCL> beep 440 1000
```

should play an A for a second. Then, in NitrOS-9, **beep** rings the bell, every key clicks (**click off** stops that), and **tetris** plays music.

A.3 How It Works

A timer interrupt on the RP2350B's first core runs 31,250 times a second. Each time, it takes any bytes the 6309 has written to the SN76489, works out the next sample of the three tone voices, the noise voice, and the keyclick or bell, adds them, and sets the PWM level. The 6309's writes reach it through a queue, so the core that runs the bus never waits.

How the Software Was Tested B

The C programs were written so that the same source builds for OS-9 with CMOC and for a Unix computer with an ordinary C compiler. That let each one be tested against a reference on the big computer, and then on the board.

B.1 zm

- CZECH, the Comprehensive Z-machine Emulation Checker, compiled for version 3 with Inform: all 349 of its tests pass, on the Unix build and on the TFR/911H's 6309, where the run takes about five seconds. They pass too with memory cut to 8K, which makes nearly every read a page from the file.
- Zork I, II, and III and Inform's Adventure: transcripts of the same commands give the same text, word for word, as **dfrotz**, a widely used interpreter, with the story wholly in memory and with only 16K to 40K for it.
- On the board: Zork I answers each command in 0.4 to 0.9 seconds; `save`, `restore`, `restart` work.

B.2 vi

- 123 editing commands—motions, operators, counts, undo, `.`, searches, `:s` and ranges—were typed into Vim and into **vi**, on a pseudo-terminal at the same pace, and the files they wrote compared: all the same.
- 27 sessions on a 300-line file with tabs and long lines checked that what **vi** shows is what is in the file, after scrolling, sideways scrolling, and edits.
- On the board: moving and typing answer in under 0.05 seconds, scrolling in 0.15, and replacing 178 strings on 116 lines in 1.8.

B.3 tsh

- Built for the Unix computer, **tsh** was driven through a pseudo-terminal and the screen read back with a terminal emulator: 68 checks of the editing keys, the history keys and the history file, completion of commands and files, sideways scrolling at 80 and 120 columns, the path, **cd** and the prompt, aliases, `;`, `&` and **w**, an rc file, and commands from a pipe.
- On the board, booted from a copy of the tether's disk with a directory on the path, 44 checks the same way: among them a module on the path that runs under its own name and is unlinked afterwards, a procedure file on the path, Ctrl/C stopping a command that never uses the terminal, **mfree** the same after running commands, and the history kept across leaving and starting **tsh**. Booted from the FujiNet, a shorter run. A command takes half a second from Return to the next prompt, completing a command's name about one second, and a file's name half a second.
- Its first runs on the board found a fault in the firmware. A packet to the tether with a run of exactly 254 bytes that are not zero, then a zero, lost the zero: the COBS encoder took it for the end of a full block, which has none. The history file, growing a line at a time, made a sector like that, and the tether stopped at the short packet. Fixed in `firmware/one/util/cobs.h`; the same sectors, copied on the board, then came through whole.
- CMOC made `err ? -1 : path`, with `path` a byte, a byte expression: 255, not −1. So every failed open looked like path 255, until it was written as two statements.

B.4 info

Its report was compared with what NitrOS-9's own **procs**, **mdir**, **mfree**, and **free** say, on the board, and with a program running in the background.

B.5 fnnews and fnstocks

- Built for the Unix computer with `fujinet-host.c`, both were driven through a pseudo-terminal, against the real news server and Finnhub, and every screen read back with a terminal emulator, reverse video and all: the topics, headlines, story pages and their information, going to a page; stocks added by symbol and by lookup, company information, deleting, and the list kept between runs.
- The same two programs were then built on fujinet-lib for NitrOS-9, and their sessions run again on the board, booted both from the tether's disk and from the FujiNet; a session booted from the FujiNet is what showed the FujiNet restarting (see Chapter 2).
- `term_printf`'s formatting gives what the C library's `printf` gives, for each conversion the programs use.
- On the board, through the FujiNet, the same sessions: a page of headlines comes in 1.7 seconds, a page of a story in 1.2, and a quote, with the list saved, in 2.2. The FujiNet's log shows the app keys read and written, as `/FujiNet/09010101.key` and `09010102.key` on its SD card. **mfree** shows all the memory back afterwards.

B.6 fnbounce

- On the board, through the FujiNet, against a server of version 2.3.1 on the Internet: the first screen, with the server and the name kept in app keys; the kinds of shape; the world with and without its information; the clients' names; reverse video; the flash and the sound turned on; and quitting with `Q` and `Esc`, and with `Esc` at the first screen. Every screen was read back with a terminal emulator.
- Built for the Unix computer, against the same server: the title screen, the settings typed and kept, the world at 40 and at 80 places, with the width kept from one run to the next, and the kinds of shape laid out for each width.
- Its speed was measured part by part. An `SS.Fuji` exchange takes 3 milliseconds, a request for the shapes on the screen 80 to 100, and drawing a frame 83; the terminal takes some 4,000 characters a second. Drawn as at first, comparing the whole copy of the screen and writing through SCF, it showed 2.8 frames a second; it now shows 9.3 of the server's 10.

B.7 fnweather

- Built for the Unix computer, against Open-Meteo and ip-api.com: the weather where the computer is, in both units; the forecast's two pages; the information page; a city looked up (London, in the rain); and the place, the units, and the weather kept in the cache from one run to the next.
- On the board, through the FujiNet, the same, with every screen read back with a terminal emulator, in color, and compared with the Unix build's; the week's chances of rain and UV index were checked against what Open-Meteo itself gives. With no cache, finding the place and fetching the weather took 17 seconds; fetching again with `R`, 18 to 19 each time; London, looked up by name, 21. Started again with the cache, **fnweather** had the weather on the screen in 2 seconds.
- The board is what showed the two ways of CMOC's in Chapter 2's *Building and testing*: at first the stack overflowed at once, and then the big icons were drawn from the wrong rows.

B.8 fnbattleship

- Built for the Unix computer, against the public server: the title, a name typed and kept, how to play, the tables, a table's players getting ready, placing ships (turned with `R`), and turns aimed with the arrows and with typed coordinates; a whole game against a robot, to its end, the menu, leaving the table, and quitting; and a table of

four, a quarter of the screen each. A program played the games, reading the screen with a terminal emulator: it fired next to its hits, and elsewhere on a checkerboard. Every screen was read back in color.

- On the board, through the FujiNet, the same whole game, twice: from typing **fnbattleship** to the list of tables in ten seconds, and then 57 shots, which won, and 49, which lost. From firing to the next turn, the robot's shot and both animations between, took 3.1 to 4.8 seconds, 3.2 or 3.3 most often; the Unix build, which runs **curl** for each request, took twice as long.
- A turn left to run out: the last twenty seconds counted down at the bottom right, and at 250, the server moved for the player and the game went on. Upstream's program had not asked the server again, and waited out the turn over and over (see Chapter 2).
- The board's screens showed a flaw, which the Unix build's had too: a ship standing upright, whose ends were half blocks across, looked a square shorter than it was. Its ends are narrower now, but whole squares.

B.9 The Firmware

Each change was tried on the board: NitrOS-9 boots, the disk and clock work, Tetris's writes reach the sound chip (182 of them in one game, by the firmware's count), and the arrow keys arrive as they should with bit 7 of `$FF42` clear and set.